

GH-200

GITHUB ACTIONS CERTIFICATION

Automação, segurança e entrega contínua

Preparação para os cinco domínios do exame, com exemplos de workflows, laboratórios guiados, consulta rápida e simulado comentado.

Erik Nathan eriknathan.me

ESTUDO, PRÁTICA E REVISÃO

05

Domínios do exame

05

Laboratórios guiados

25

Questões comentadas

Sumário

CAPÍTULO 1 – COMO ESTUDAR E ENTENDER O EXAME

1.1	Objetivo do material	4	1.4	Sequência de estudo sugerida	5
1.2	Mapa dos cinco domínios	4	1.5	Convenções dos exemplos	5
1.3	Condições da prova	4			

CAPÍTULO 2 – DOMÍNIO 1 – CRIAR E GERENCIAR FLUXOS DE TRABALHO

2.1	Modelo de execução	6	2.9	Variáveis de ambiente padrão	11
2.2	Primeiro workflow completo	6	2.10	Scripts do repositório	12
2.3	Eventos e filtros	7	2.11	Matrizes	12
2.4	Execução manual e entradas tipadas	8	2.12	Jobs e serviços em contêiner	13
2.5	Agendamento	8	2.13	Âncoras, aliases e mesclagem YAML	13
2.6	Dependências, condições e falhas	9	2.14	Cache, artefatos e resumos	14
2.7	Contextos e momento da avaliação	10	2.15	Publicar pacotes e imagens	15
2.8	Troca de dados	10	2.16	Decisão rápida do domínio	16

CAPÍTULO 3 – DOMÍNIO 2 – CONSUMIR E SOLUCIONAR PROBLEMAS DE FLUXOS DE TRABALHO

3.1	Ler uma execução de fora para dentro	17	3.6	Desabilitar, excluir arquivo e excluir execução	19
3.2	Tabela de diagnóstico	17	3.7	Escolher a forma de reutilização	19
3.3	Anatomia da página de execução	18	3.8	Contrato de um workflow reutilizável	19
3.4	Logs, debug e reexecução	18	3.9	Decisão rápida do domínio	21
3.5	Operar pela CLI e pela API	18			

CAPÍTULO 4 – DOMÍNIO 3 – CRIAR E MANTER AÇÕES

4.1	Escolher o tipo de ação	22	4.5	Ações Docker	24
4.2	Estrutura de metadados	22	4.6	Testar e distribuir	25
4.3	Ação composta com entrada e saída	22	4.7	Versionamento, releases e imutabilidade	27
4.4	Ação JavaScript mínima	24	4.8	Decisão rápida do domínio	27

CAPÍTULO 5 – DOMÍNIO 4 – GERENCIAR O GITHUB ACTIONS PARA A ORGANIZAÇÃO

5.1	Governança em camadas	29	5.6	Segredos, variáveis e escopo	32
5.2	Templates organizacionais	29	5.7	Uso de segredos	32
5.3	Runners hospedados e auto-hospedados	30	5.8	Administração por CLI e REST	33
5.4	Labels e grupos	30	5.9	Exemplo de desenho organizacional	33
5.5	Operação, rede e imagens	31	5.10	Decisão rápida do domínio	34

CAPÍTULO 6 – DOMÍNIO 5 – AUTOMAÇÃO SEGURA E OTIMIZADA

6.1	Identidade, permissão e confiança	35	6.6	Ambientes e aprovações	37
6.2	Permissões do GITHUB_TOKEN	35	6.7	Atestados e proveniência	37
6.3	Entradas não confiáveis e injeção	36	6.8	Otimização de tempo e custo	38
6.4	Pull requests e fronteiras de privilégio	36	6.9	Retenção e administração programática	38
6.5	OIDC para credenciais temporárias	36	6.10	Decisão rápida do domínio	38

CAPÍTULO 7 – LABORATÓRIOS GUIADOS

7.1	Laboratório de pipeline: matriz, outputs, arte-fatos e falha controlada	40	7.4	Laboratório de governança	43
7.2	Laboratório de reutilização	42	7.5	Laboratório de revisão de segurança e eficiência	43
7.3	Laboratório de autoria de ação	42			

CAPÍTULO 8 – CONSULTA RÁPIDA E PEGADINHAS

8.1	Números e limites	44	8.4	Afirmações que exigem contexto	46
8.2	Sintaxe que precisa reconhecer	44	8.5	Comandos de workflow	46
8.3	Diferenças que mudam a resposta	45			

CAPÍTULO 9 – ÍNDICE DE TEMAS DOS SIMULADOS

CAPÍTULO 10 – SIMULADO AUTORAL – 25 QUESTÕES

10.1	Questões do domínio 1	49	10.5	Questões do domínio 5	54
10.2	Questões do domínio 2	50	10.6	Gabarito comentado	55
10.3	Questões do domínio 3	51	10.7	Registro de erros	56
10.4	Questões do domínio 4	53			

CAPÍTULO 11 – REVISÃO FINAL E FONTES

11.1	Checklist de prontidão	57	11.3	Fontes oficiais para manutenção do material	58
11.2	Perguntas para revisão oral	57	11.4	Como atualizar este guia	58

Material desenvolvido a partir do roteiro de estudos fornecido, com explicações, exemplos e exercícios originais. Objetivos do exame e pontos sujeitos a mudança conferidos nas fontes oficiais em **30 de setembro de 2026**. Os exemplos têm como referência o **GitHub.com**; recursos e versões de ações podem diferir no GitHub Enterprise Server (GHES).

CAPÍTULO 1

Como estudar e entender o exame

1.1 Objetivo do material

O GitHub Actions automatiza tarefas a partir de eventos: testar um pull request, gerar um pacote, implantar uma aplicação ou executar manutenção. Para a GH-200, saber escrever YAML é apenas parte da preparação. Você também precisa prever uma execução, diagnosticar falhas, criar ações e administrar a plataforma com segurança.

Estude cada tema em três etapas: explique o conceito, execute um exemplo e modifique uma condição para prever o resultado. Se consegue repetir o procedimento, mas não explicar por que funciona, retorne ao conceito antes de avançar.

1.2 Mapa dos cinco domínios

Domínio	Peso oficial	Competência central	Evidência prática de domínio
Criar e gerenciar fluxos de trabalho	20–25%	Eventos, estrutura, dados e execução	Construir um workflow com matriz, dependências, outputs e artefatos
Consumir e solucionar problemas de fluxos de trabalho	15–20%	Interpretar execuções e reutilizar automações	Encontrar a causa de uma falha e escolher a forma de reutilização
Criar e manter ações	15–20%	Metadados, implementação e distribuição	Criar uma ação com entrada, saída e tratamento de erro
Gerenciar o GitHub Actions para a organização	20–25%	Políticas, runners, segredos e variáveis	Desenhar acesso e governança para vários repositórios
Automação segura e otimizada	10–15%	Confiança, credenciais e eficiência	Revisar permissões, OIDC, dependências e custo de execução

As faixas acima vêm das habilidades medidas a partir de janeiro de 2026. Elas orientam a distribuição do estudo, mas não representam uma quantidade fixa de questões por domínio. Guia oficial GH-200.

Em materiais em inglês, os domínios aparecem como *Author and Manage Workflows*, *Consume and Troubleshoot Workflows*, *Author and Maintain Actions*, *Manage GitHub Actions for the Enterprise* e *Secure and Optimize Automation*. Materiais antigos podem agrupar o conteúdo em quatro domínios; classifique seus erros usando os cinco grupos atuais.

1.3 Condições da prova

Na consulta de 30/09/2026, a página da certificação informa **100 minutos de avaliação**, disponibilidade em português do Brasil e **validade de dois anos**. A página também descreve uma transição no processo de recertificação. Consulte as condições vigentes ao agendar. Página da certificação.

Não use quantidade de questões, preço ou percentual de acertos de um simulado como garantia das condições do exame. Uma pontuação escalonada não equivale diretamente à porcentagem de respostas corretas.

1.4 Sequência de estudo sugerida

Etapa	Atividade	Entrega pessoal
Diagnóstico	Fazer a avaliação prática e ler o mapa do exame	Lista de lacunas por domínio
Fundamentos	Criar workflows, interpretar eventos e trocar dados	Pipeline simples funcionando
Reutilização e depuração	Extrair um workflow reutilizável e provocar uma falha	Registro da causa e da correção
Autoria de ações	Criar e testar uma ação composta	<code>action.yml</code> com contrato documentado
Administração	Desenhar políticas, ambientes e runners	Diagrama ou tabela de acesso
Segurança e eficiência	Revisar token, entradas, dependências e custo	Lista de melhorias justificadas
Revisão	Resolver as questões deste material e refazer o diagnóstico	Explicação dos erros sem consultar o gabarito

Os quatro simulados de 60 questões mencionados no roteiro original não foram fornecidos. Este material inclui um **simulado autoral de 25 questões**, com cinco questões por domínio, e uma tabela para registrar avaliações adicionais. Ele serve à revisão; sua distribuição uniforme não reproduz os pesos do exame.

1.5 Convenções dos exemplos

- Um bloco identificado por um caminho completo, como `.github/workflows/ci.yml`, representa um arquivo. Blocos descritos como **trecho** precisam ser inseridos no contexto indicado.
- Os exemplos usam Bash em runners Ubuntu, salvo indicação contrária.
- Nomes como `ORG`, `REPO`, `RUN_ID` e caminhos de scripts de implantação são valores a substituir.
- As tags das ações facilitam a leitura dos exercícios. Em automações protegidas, resolva uma versão revisada para seu **SHA completo** e mantenha as atualizações sob revisão.
- Foram consultadas as referências de `checkout`, `setup-node`, `upload-artifact` e `download-artifact`. Os exemplos usam, respectivamente, `v7`, `v7`, `v7` e `v8`; esses números são versões das ações, não versões da aplicação nem objetivos de memorização.

Domínio 1 — Criar e gerenciar fluxos de trabalho

2.1 Modelo de execução

Elemento	Função	Exemplo
Evento	Dispara a automação	<code>push</code> , <code>pull_request</code> , <code>schedule</code>
Workflow	Define a automação em <code>.github/workflows/*.yaml</code> ou <code>*.yaml</code>	Pipeline de integração contínua
Job	Agrupar trabalho executado em um runner	Testes, build, implantação
Step	Executa um comando ou uma ação dentro do job	<code>run: npm test</code>
Action	Encapsula uma operação reutilizável	<code>uses: actions/checkout@v7</code>
Runner	Executa o job	Uma máquina hospedada pelo GitHub ou administrada pela organização

Jobs independentes podem executar em paralelo. Dentro do modelo usual de um job, os steps são executados na ordem declarada. Um step pode criar arquivos usados pelo seguinte no mesmo workspace, mas cada comando `run` inicia seu próprio processo de shell: um simples `export` não é um mecanismo de persistência entre steps.

Pense em três perguntas ao ler um YAML: **o que dispara**, **onde executa** e **de que depende**. Um workflow com sintaxe válida ainda pode não executar por causa de filtros, políticas ou falta de runner.

2.2 Primeiro workflow completo

Arquivo `.github/workflows/primeiro.yaml`:

```
name: Primeiro workflow

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  verificar:
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    steps:
      - name: Obter os arquivos
        uses: actions/checkout@v7
        with:
          persist-credentials: false

      - name: Verificar o checkout
        run: git rev-parse --is-inside-work-tree

      - name: Registrar o resultado
        run: printf 'Checkout verificado.\n' >> "$GITHUB_STEP_SUMMARY"
```

`name` organiza a interface; `on` escolhe os eventos; `permissions` limita o token; `runs-on` seleciona o ambiente. `uses` chama uma ação e `run` executa código. `with` fornece entradas à ação; não é um bloco de variáveis de ambiente. Sintaxe de workflows.

2.3 Eventos e filtros

Evento	Uso típico	Atenção ao interpretar
<code>push</code>	Validar alterações em branches ou tags	<code>branches</code> e <code>tags</code> selecionam referências diferentes
<code>pull_request</code>	Testar uma proposta de alteração	O filtro <code>branches</code> se refere à branch de destino
<code>pull_request_target</code>	Automatizar metadados do PR no contexto da base	Exige cuidado especial com código vindo do PR
<code>workflow_dispatch</code>	Execução manual por interface, CLI ou API	O workflow precisa existir na branch padrão para receber esse evento
<code>repository_dispatch</code>	Receber um evento externo via API	O tipo personalizado entra em <code>types</code>
<code>schedule</code>	Rotinas periódicas	Executa o commit atual da branch padrão
<code>workflow_call</code>	Ser chamado por outro workflow	Define uma interface de entradas, segredos e saídas
<code>workflow_run</code>	Reagir a outro workflow	Considere a fronteira de confiança entre as duas execuções
<code>release</code>	Automatizar uma publicação	<code>types: [published]</code> seleciona a atividade
<code>merge_group</code>	Validar a fila de merge	Necessário quando a estratégia de integração usa merge queue e checks correspondentes
<code>issues</code>	Reagir à abertura ou edição de uma issue	<code>types: [opened, edited, milestoned]</code> ; não cobre comentários
<code>issue_comment</code>	Reagir a comentário em issue ou PR	Evento separado de <code>issues</code> ; <code>types: [created]</code> para novos comentários
<code>discussion</code>	Automatizar discussões	<code>types: [created, edited, answered]</code>
<code>create</code>	Criação de branch ou tag	Não existe evento <code>tag</code> ; a criação de tag entra em <code>create</code>
<code>check_run</code>	Reagir a um check individual	<code>types: [created, completed]</code>
<code>check_suite</code>	Reagir a um conjunto de checks	Só dispara se o arquivo de workflow estiver na branch padrão
<code>branch_protection_rule</code>	Auditar mudanças em regras de proteção	<code>types: [created, edited, deleted]</code> ; para excluir uma atividade, liste apenas as desejadas

Consulte os detalhes de cada evento, inclusive a referência e o commit associados. Eventos que disparam workflows.

Nem toda atividade do repositório é um evento de workflow. Convidar alguém para o repositório, por exemplo, não dispara execuções.

Não existe chave de exclusão de atividades. `types` é uma lista de inclusão: para rodar em criação e edição mas não em exclusão, escreva `types: [created, edited]`. Formas como `notTypes`, `type`, `when` ou `filter` não fazem parte da sintaxe.

Trecho de configuração:

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]
    paths:
      - 'src/**'
```

- 'package*.json'
- '.github/workflows/**'

Para esse workflow executar, o PR precisa atender tanto ao filtro de branch quanto ao de caminhos. Quando `branches` / `branches-ignore` e `paths` / `paths-ignore` aparecem juntos, a relação é **E**, não **ou**: os dois filtros precisam ser satisfeitos. `branches` e `branches-ignore` não podem coexistir para o mesmo evento — use `branches` quando precisar incluir e excluir padrões no mesmo filtro.

`types` filtra a atividade do evento. Adicionar um label não corresponde a `synchronize`: esse tipo representa atualização dos commits do PR.

Padrões de branch aceitam curingas. `branches: ['feature/*']` seleciona qualquer branch iniciada por `feature/`. Para reagir a push em qualquer branch e também à criação de branch ou tag, combine eventos em lista: `on: [push, create]`. A lista usa colchetes; chaves definem mapeamento em YAML e não funcionam aqui.

Evite configurar um check obrigatório de modo que seus filtros impeçam a execução esperada. Ao depurar um check pendente, verifique primeiro se o evento realmente selecionou o workflow.

2.4 Execução manual e entradas tipadas

Trecho para o campo `on`:

```
on:
  workflow_dispatch:
    inputs:
      alvo:
        description: Ambiente de destino
        type: choice
        options: [teste, homologacao]
        default: teste
        required: true
      publicar:
        description: Publicar o resultado
        type: boolean
        default: false
      tentativas:
        description: Quantidade máxima de tentativas
        type: number
        default: 2
```

`workflow_dispatch` admite `boolean`, `choice`, `number`, `environment` e `string`. Um input do tipo `environment` seleciona um nome; o job ainda precisa referenciá-lo em `environment` para aplicar suas proteções. O contexto `inputs` preserva booleanos; `github.event.inputs` representa os valores como strings.

Exemplo de condição: `if: ${inputs.publicar}`. Comparar um booleano a `'true'` pode introduzir conversões desnecessárias. Também não confunda tipo com validação de negócio: um número pode precisar de uma verificação de faixa.

A execução manual acontece pela aba Actions, pelo GitHub CLI ou pela REST API. Disparar o workflow de um repositório **privado** pela REST API exige autenticação por **personal access token**: chave SSH serve para operações Git, não para a API, e usuário e senha não são aceitos. O valor do disparo manual está em testar e depurar de forma controlada e intencional — não em acelerar a execução nem em restringir branches, que dependem dos filtros do evento.

2.5 Agendamento

Trecho para uma rotina nos dias úteis:

```

on:
  schedule:
    - cron: '17 9 * * 1-5'
      timezone: America/Recife

```

Os cinco campos são minuto, hora, dia do mês, mês e dia da semana. Sem `timezone`, o padrão é UTC; a documentação atual permite um fuso IANA. O menor intervalo é cinco minutos. Agendamento não garante início no segundo exato: carga da plataforma pode atrasar execuções. Referência de `schedule`.

2.6 Dependências, condições e falhas

`needs` declara dependência entre jobs. Se `publicar` depende de `testar`, ele aguarda a conclusão de `testar`. Por padrão, uma falha ou omissão em uma dependência impede os jobs dependentes de prosseguir, salvo uma condição que trate esse estado.

Expressão	Uso
<code>success()</code>	Prosseguir quando os steps anteriores tiveram sucesso
<code>failure()</code>	Executar diagnóstico após uma falha
<code>cancelled()</code>	Identificar cancelamento
<code>always()</code>	Avaliar como verdadeiro inclusive em cancelamentos
<code>!cancelled()</code>	Permitir trabalho de encerramento sem continuar após cancelamento
<code>needs.testar.result</code>	Ler o resultado de um job dependência
<code>steps.teste.outcome</code>	Resultado do step antes de aplicar <code>continue-on-error</code>
<code>steps.teste.conclusion</code>	Resultado do step após aplicar <code>continue-on-error</code>

Um `if` sem uma função de status normalmente recebe a condição implícita de sucesso. Por isso, `testar` apenas um output nem sempre faz um step executar depois de uma falha. `continue-on-error` aceita a falha no ponto configurado; não torna o comando correto. Expressões e funções de status.

Trecho de um job destinado a registrar resultados:

```

jobs:
  registrar:
    needs: [testar, analisar]
    if: ${!cancelled()}
    runs-on: ubuntu-24.04
    steps:
      - env:
          RESULTADO_TESTES: ${needs.testar.result}
          RESULTADO_ANALISE: ${needs.analisar.result}
        run: |
          printf 'Testes: %s\n' "$RESULTADO_TESTES"
          printf 'Análise: %s\n' "$RESULTADO_ANALISE"

```

O trecho pressupõe que `testar` e `analisar` existem no mesmo workflow. Ele registra seus estados; não converte uma falha de teste em aprovação para implantação.

2.7 Contextos e momento da avaliação

Contexto	Informação	Exemplo
github	Evento, repositório, referências e execução	github.event_name, github.ref, github.sha
runner	Sistema e diretórios do runner	runner.os, runner.temp
env	Variáveis definidas no workflow, job ou step	env.MODO
vars	Configuração mantida no GitHub	vars.REGIAO
secrets	Valores sensíveis disponíveis no escopo	secrets.API_TOKEN
inputs	Entradas de execução manual ou reutilização	inputs.alvo
matrix	Combinação atual da matriz	matrix.node
strategy	Metadados da estratégia	strategy.job-index
needs	Outputs e resultados de dependências diretas	needs.build.outputs.versao
steps	Outputs e resultados dos steps identificados	steps.meta.outputs.versao
job	Estado e serviços do job atual	job.status, job.services

O GitHub avalia expressões em fases diferentes. Condições de job e seleção de runner podem ser decididas antes de existir um shell. Variáveis como `$GITHUB_WORKSPACE` são usadas no runner; não substituem automaticamente expressões em qualquer campo do YAML. A disponibilidade de cada contexto depende da chave que o utiliza. Referência de contextos.

Use strings entre aspas simples dentro de expressões: `${{ github.ref == 'refs/heads/main' }}`. Um output de step é texto; converta JSON quando precisar de um booleano, número ou estrutura, por exemplo `fromJson(needs.-preparar.outputs.matriz)`.

2.8 Troca de dados

Necessidade	Mecanismo	Escopo
Variável para comandos seguintes	GITHUB_ENV	Próximos steps do mesmo job
Diretório adicional no caminho de executáveis	GITHUB_PATH	Próximos steps do mesmo job
Resultado nomeado de um step	GITHUB_OUTPUT	Consumido pelo contexto <code>steps</code>
Resultado entre jobs	jobs.<id>.outputs e needs	Jobs dependentes
Resultado de um workflow reutilizável	on.workflow_call.outputs	Workflow chamador
Arquivos de build ou relatórios	Artefatos	Jobs e pessoas com acesso
Relatório legível na execução	GITHUB_STEP_SUMMARY	Interface da execução

Trecho de steps de um mesmo job:

```
steps:
  - id: meta
    run: |
      printf 'versao=lab-%s\n' "$GITHUB_RUN_NUMBER" >> "$GITHUB_OUTPUT"
      printf 'MODO=estudo\n' >> "$GITHUB_ENV"
  - env:
      VERSAO: ${{ steps.meta.outputs.versao }}
```

```
run: |
  printf 'Versão: %s; modo: %s\n' "$VERSAO" "$MOD0"
  printf 'Versão produzida: `%s`\n' "$VERSAO" >> "$GITHUB_STEP_SUMMARY"
```

O step que escreve em `GITHUB_ENV` não recebe automaticamente o novo valor em seu próprio processo. O próximo step recebe. As variáveis padrão `GITHUB_*` e `RUNNER_*` não podem ser redefinidas por esse arquivo. Para valores multilinha, use o formato de delimitadores documentado e garanta que o delimitador não ocorra no conteúdo. Comandos de workflow e arquivos de ambiente.

`GITHUB_ENV` atravessa steps do mesmo job, mas não atravessa jobs. Um job seguinte começa com o ambiente limpo; para passar valores adiante, use output de job com `needs`.

2.9 Variáveis de ambiente padrão

O GitHub define um conjunto de variáveis disponíveis em todos os steps, sem declaração. Elas são sempre em maiúsculas.

Variável	Conteúdo
<code>GITHUB_REPOSITORY</code>	<code>owner/repositorio</code>
<code>GITHUB_REF</code>	Referência completa, como <code>refs/heads/main</code>
<code>GITHUB_SHA</code>	Commit da execução
<code>GITHUB_WORKSPACE</code>	Diretório do checkout
<code>GITHUB_ACTOR</code>	Conta que originou a execução
<code>GITHUB_RUN_NUMBER</code> / <code>GITHUB_RUN_ID</code>	Identificação da execução
<code>GITHUB_EVENT_NAME</code>	Evento que disparou a execução
<code>GITHUB_ACTIONS</code>	Sempre <code>true</code> quando o código roda no Actions; útil para diferenciar da execução local
<code>RUNNER_OS</code>	<code>Linux</code> , <code>Windows</code> ou <code>macOS</code>
<code>RUNNER_ARCH</code>	Arquitetura do processador
<code>RUNNER_TEMP</code> / <code>RUNNER_TOOL_CACHE</code>	Diretórios temporário e de ferramentas

Três regras que mudam respostas:

- Variáveis padrão **não** são lidas pelo contexto `env`. Elas existem no runner e usam a sintaxe do shell: `$GITHUB_REPOSITORY` em Bash, `$env:GITHUB_REPOSITORY` em PowerShell. O que usa `${{ ... }}` é o contexto correspondente, como `${{ github.repository }}`.
- Você não pode criar uma variável em `env`: reutilizando um nome padrão. Declarar `env: GITHUB_REPOSITORY:` ou `tra/coisa` é inválido.
- `JAVA_HOME` e semelhantes vêm do tool cache do runner, não são variáveis padrão do Actions, e podem ser redefinidas normalmente.

Trate nomes de variáveis como sensíveis a maiúsculas e minúsculas, independentemente do sistema operacional do runner. Prefira variáveis padrão a caminhos fixos ao referenciar o sistema de arquivos: `${{ github.workspace }}` so-brevia a mudanças de estrutura que um caminho escrito à mão não sobrevive.

2.10 Scripts do repositório

Um script versionado é executado pelo `run`, informando o caminho a partir do workspace. Defina `working-directory` quando vários steps compartilham a mesma pasta, em vez de repetir o caminho.

```
steps:
  - uses: actions/checkout@v7
  - run: ./scripts/validar.sh
    working-directory: ferramentas
```

O arquivo precisa ter permissão de execução **antes** de ser usado. Isso é propriedade do arquivo no Git, não do YAML: rode `chmod +x scripts/validar.sh` e faça o commit, ou aplique `chmod +x` em um step anterior no próprio runner. Não existe palavra-chave de workflow que torne um arquivo executável, e `sudo` não resolve — `sudo` eleva privilégio, não acrescenta o bit de execução. O mesmo vale para o `entrypoint.sh` de uma ação Docker.

2.11 Matrizes

Uma matriz expande combinações de configurações. Dois sistemas e duas versões geram quatro variantes antes de aplicar exclusões ou inclusões.

Trecho de job:

```
jobs:
  testar:
    name: Testes - ${matrix.os} - Node ${matrix.node}
    runs-on: ${matrix.os}
    strategy:
      fail-fast: false
      max-parallel: 2
      matrix:
        os: [ubuntu-24.04, windows-2025]
        node: ['22', '24']
        exclude:
          - os: windows-2025
            node: '22'
        include:
          - os: ubuntu-24.04
            node: '24'
            teste_extra: true
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
        with:
          node-version: ${matrix.node}
          package-manager-cache: false
      - run: node --version
```

Aqui ficam três variantes. O `include` acrescenta uma propriedade à combinação compatível; não cria necessariamente um job adicional. Quando uma inclusão não pode ser combinada sem substituir os valores originais, ela pode gerar uma nova combinação.

`fail-fast: false` permite investigar outras variantes após uma falha. `max-parallel: 2` limita simultaneidade, mas não reduz a quantidade total de jobs. Estratégias de matriz.

2.12 Jobs e serviços em contêiner

`container` muda o ambiente em que os steps do job executam. `services` cria dependências, como banco de dados ou fila. Isso difere de uma ação Docker, que empacota uma ação específica em uma imagem.

Arquivo `.github/workflows/postgres.yml`:

```
name: Verificar serviço PostgreSQL
on: workflow_dispatch
permissions: {}

jobs:
  banco:
    runs-on: ubuntu-24.04
    container: postgres:17
    services:
      postgres:
        image: postgres:17
        env:
          POSTGRES_USER: aluno
          POSTGRES_PASSWORD: senha-apenas-do-laboratorio
          POSTGRES_DB: estudo
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - name: Consultar o serviço
        env:
          PGPASSWORD: senha-apenas-do-laboratorio
        run: psql -h postgres -U aluno -d estudo -c 'SELECT 1;'
```

Dentro da rede dos contêineres, `postgres` resolve para o serviço. Se os steps rodassem diretamente no runner, você publicaria a porta com `ports` e usaria `localhost` e a porta publicada. O healthcheck evita que o cliente tente acessar um banco ainda inicializando. Esse uso exige runner Linux com Docker. Serviços PostgreSQL.

A senha acima é exclusiva de um banco descartável do exercício. Credenciais de sistemas reais devem usar o mecanismo de segredos ou uma integração de identidade apropriada.

2.13 Âncoras, aliases e mesclagem YAML

Uma âncora `&nome` identifica um valor; um alias `*nome` reutiliza esse valor dentro do documento. Isso reduz repetição sem criar uma interface de inputs ou um componente compartilhado entre repositórios.

Arquivo `.github/workflows/ancoras.yml`:

```
name: Reutilizar configuração local
on: workflow_dispatch
permissions: {}

jobs:
  primeiro:
    runs-on: ubuntu-24.04
    env: &config_estudo
      MOD0: laboratorio
      NIVEL_LOG: info
    steps:
      - run: printf '%s\n' "$MOD0"
```

```
segundo:
  runs-on: ubuntu-24.04
  env: *config_estudo
  steps:
    - run: printf '%s\n' "$NIVEL_LOG"
```

Duas regras de sintaxe YAML aparecem em questões conceituais. O YAML é um superconjunto estrito do JSON, com o acréscimo de quebras de linha e indentação sintaticamente significativas, como em Python. Diferente do Python, porém, **o YAML não aceita caractere literal de tabulação para indentação** — apenas espaços. O YAML também admite comentários, que o JSON não admite.

O suporte a âncoras e aliases está documentado pelo GitHub. O programa GH-200 também menciona mapeamentos mesclados com `<<`. Em YAML que aceita essa construção, `<<: *base` incorpora chaves de um mapeamento e permite especialização. **Não trate um alias simples como uma mesclagem.** Os exemplos executáveis deste material usam âncoras e aliases; a documentação de reutilização consultada não apresenta `<<` como recurso garantido de workflows. Ao encontrar essa sintaxe, explique sua intenção e valide o suporte no ambiente utilizado. Referência de reutilização, objetivos do exame.

2.14 Cache, artefatos e resumos

Recurso	Pergunta que responde	Exemplo
Cache	O que posso reaproveitar para economizar trabalho?	Download de dependências
Artefato	Qual arquivo esta execução produziu?	Pacote, cobertura, evidência de teste
Output	Qual pequeno valor preciso passar adiante?	Versão, caminho, identificador
Resumo	O que a pessoa precisa entender sobre a execução?	Resultado, métricas e links

O pipeline deve continuar correto quando não encontra cache. Não coloque segredos em diretórios cacheados, nem use um cache como única cópia de um artefato de release. Conceito de cache.

Em um projeto Node que já possui `package.json` e `package-lock.json`, este trecho reutiliza downloads do npm:

```
steps:
  - uses: actions/checkout@v7
  - uses: actions/setup-node@v7
    with:
      node-version: '24'
      cache: npm
      cache-dependency-path: package-lock.json
  - run: npm ci
  - run: npm test
```

O cache do `setup-node` não elimina a instalação e não é uma cópia de `node_modules`. Em monorepos, aponte para os lockfiles corretos. Mudanças no lockfile precisam afetar a identidade do cache. Cache no `setup-node`.

Ao publicar artefatos em uma matriz, inclua os eixos no nome para evitar colisões. Use `if-no-files-found: error` quando a ausência de arquivos deve reprovar o job. A retenção do artefato respeita as políticas aplicáveis. Upload de artefatos.

Quando você usa a ação de cache diretamente, `key` é a chave exata e `restore-keys` é a lista de chaves alternativas, tentadas em ordem quando a chave exata não é encontrada. Em um `cache miss`, o workflow segue normalmente e a ação grava um cache novo se o job terminar com sucesso — não há interrupção nem intervenção manual. A busca

acontece no repositório atual; o cache não é procurado em outros repositórios. O output `cache-hit` informa se houve acerto, e `path` define o que é armazenado.

O artefato tem retenção própria: `retention-days` na ação de upload define o prazo daquele artefato individualmente, sem depender de configuração da organização. A data de expiração já calculada não aparece na interface — consulte o campo `expires_at` pela API de Actions. Um artefato excluído não pode ser restaurado, e excluir libera a cota de armazenamento.

Para baixar um artefato dentro de outro job da mesma execução, use `actions/download-artifact` (não existe `actions/download`). Pela interface, o artefato fica na seção **Artifacts** da página da execução — não em Releases, nem nos detalhes do job, nem no pull request. Qualquer pessoa autenticada com acesso de **leitura** ao repositório pode baixá-lo.

Um badge de status é uma referência visual, não uma proteção de branch. Exemplo de URL, substituindo os identificadores: `https://github.com/ORG/REPO/actions/workflows/ci.yml/badge.svg`. Regras que bloqueiam merge ou implantação precisam ser configuradas nos recursos correspondentes.

Dois parâmetros ajustam o que o badge mostra, acrescentados ao fim da URL:

Parâmetro	Efeito
<code>?branch=NOME-DA-BRANCH</code>	Status das execuções daquela branch
<code>?event=push</code>	Status das execuções disparadas por aquele evento

Em repositório privado, o badge não é acessível de fora. A restrição existe para impedir incorporação ou vinculação por origens não autorizadas, não porque o recurso esteja desligado.

2.15 Publicar pacotes e imagens

A publicação acontece dentro do workflow, autenticada pelo `GITHUB_TOKEN` com `packages: write`.

Para uma imagem de contêiner, o destino é o GitHub Container Registry, cujo host é `ghcr.io`. Em uma configuração, o valor costuma aparecer em `env.REGISTRY` — é ele que identifica o destino, não o nome das ações usadas.

```
env:  
  REGISTRY: ghcr.io  
  IMAGE_NAME: ${github.repository}
```

Para um pacote npm no GitHub Packages, três peças precisam coexistir: o step `actions/setup-node`, o `registry-url` apontando para `https://npm.pkg.github.com/` e a variável `NODE_AUTH_TOKEN` no step de publicação. Faltando qualquer uma, a publicação falha ou vai para o registro errado.

```
steps:  
  - uses: actions/checkout@v7  
  - uses: actions/setup-node@v7  
    with:  
      node-version: '24'  
      registry-url: 'https://npm.pkg.github.com/'  
  - run: npm ci  
  - run: npm publish  
  env:  
    NODE_AUTH_TOKEN: ${secrets.GITHUB_TOKEN}
```

2.16 Decisão rápida do domínio

Situação	Escolha	Erro a evitar
Uma tarefa só pode começar após outra	<code>needs</code>	Confiar na ordem dos jobs no arquivo
Pequeno valor entre jobs	Output do job	Esperar propagação de <code>GITHUB_ENV</code>
Arquivos entre jobs	Artefato	Supor workspace compartilhado
Variações de SO e runtime	Matriz	Multiplicar combinações sem medir utilidade
Dependência temporária de banco	<code>services</code>	Usar <code>localhost</code> dentro do contêiner errado
Repetição no mesmo YAML	Âncora e alias	Confundir com componente versionado
Instalação repetitiva de dependências	Cache com chave apropriada	Depender da presença do cache para funcionar

Domínio 2 — Consumir e solucionar problemas de fluxos de trabalho

3.1 Ler uma execução de fora para dentro

- .. Identifique o evento, a referência, o commit e o autor que originaram a execução.
- !.. Confira quais filtros selecionaram o workflow.
- !.. Leia o grafo de dependências e o estado de cada job.
- !.. Em uma matriz, identifique a combinação de SO, runtime e demais eixos.
- !.. Encontre o primeiro step que produziu um erro relevante.
- !.. Compare a configuração com a última execução bem-sucedida.

O último erro visível pode ser consequência. Por exemplo: o upload falhou porque o build não produziu arquivos; aumentar permissões do upload não corrige o erro de compilação.

3.2 Tabela de diagnóstico

Sintoma	Hipóteses iniciais	Evidência a procurar
Workflow não aparece ou não dispara	Caminho, YAML inválido, evento ou branch inadequados	Arquivo em <code>.github/workflows</code> , filtros e branch padrão
Job fica em fila	Runner offline, labels incompatíveis ou falta de capacidade	Grupos, labels, estado e políticas do runner
Job aparece como <code>skipped</code>	<code>if</code> falso ou dependência omitida/falha	Expressão e <code>needs.<job>.result</code>
Falha 403 na API	Token sem permissão ou política restritiva	Permissões efetivas e endpoint chamado
Recurso privado retorna 404	Caminho incorreto ou recurso não acessível ao token	Identidade, repositório e política de compartilhamento
Ação local não é encontrada	Checkout ausente ou caminho errado	<code>action.yml</code> , diretório de trabalho e conteúdo baixado
Comando funciona localmente, mas falha no CI	Versão, shell, permissões ou ferramenta ausente	SO, runtime, PATH e mensagens de instalação
Só uma variante falha	Diferença de SO, arquitetura ou runtime	Nome completo do job da matriz
Serviço não responde	DNS, porta, healthcheck ou credencial incorretos	Topologia: host ou contêiner; logs do serviço
Artefato não é localizado	Nome, caminho, execução ou retenção incorretos	ID da execução, expiração e sucesso do upload
Output está vazio	<code>id</code> , nome, escopo ou mapeamento ausente	Step produtor, output do job e dependência

Essas hipóteses orientam a investigação; não substituem o log. Registre a condição que reproduz o problema antes de alterar várias configurações ao mesmo tempo.

3.3 Anatomia da página de execução

A aba **Actions** lista as execuções. Diretamente na lista você vê o status, a branch em que a execução foi disparada e a duração de cada uma. A configuração **não** aparece ali: ela está no arquivo `.yaml`, alcançável pelo link da execução, pela opção **View workflow file** no menu da execução ou pelo diretório `.github/workflows` na aba Code.

Os filtros da lista respondem a perguntas diferentes:

Filtro	Pergunta
Event	Quais execuções vieram de <code>pull_request</code> , <code>push</code> , <code>schedule</code> ...?
Status	Quais falharam? O valor é <code>failure</code> — não <code>failed</code> nem <code>errored</code>
Branch	Quais rodaram em determinada branch?
Actor	Quem originou a execução?

Para uma execução disparada por pull request, o resultado aparece em três lugares: no próprio PR antes do merge, na aba **Checks** do PR e na aba Actions do repositório. A aba Issues e a aba Insights não mostram execuções.

Dentro de um job, o GitHub acrescenta dois steps que você não escreveu: **"Set up job"** no início e **"Complete job"** no fim. Em runners hospedados, o "Set up job" registra o sistema operacional, a imagem do runner com link para as ferramentas pré-instaladas, as permissões concedidas ao `GITHUB_TOKEN` e a origem dos segredos. Não há varredura de vulnerabilidades nesse step — isso seria uma ação separada no workflow.

Ao procurar texto nos logs, lembre-se de que **apenas steps expandidos entram na busca**. Um resultado vazio geralmente significa step recolhido ou texto inexistente, não log arquivado nem limite de consultas. Para citar uma linha a alguém, clique no número da linha do step e copie o link da barra de endereços: isso leva a pessoa ao ponto exato, com o contexto ao redor preservado.

Excluir arquivos de log exige acesso de **escrita** ao repositório. Baixar artefatos exige apenas **leitura**. Perfis `admin` e `owner` também conseguem, mas a pergunta relevante costuma ser qual é o nível necessário, não qual é suficiente.

Nos bastidores, o Actions publica status, resultados e logs pela **Checks API**: cada execução de workflow cria um *check suite*, que contém um *check run* por job, e cada job contém seus steps. A Statuses API existe para status de commit vindos de serviços externos; a Actions API serve para operar workflows e execuções.

3.4 Logs, debug e reexecução

`ACTIONS_STEP_DEBUG=true` habilita mensagens detalhadas dos steps; `ACTIONS_RUNNER_DEBUG=true` acrescenta diagnóstico do runner. Se uma dessas opções estiver configurada como segredo e variável, o segredo tem prioridade. Ative apenas durante a investigação e revise o conteúdo antes de compartilhar logs. Logs de debug.

Uma reexecução usa o mesmo `GITHUB_SHA` e `GITHUB_REF` da execução original. Portanto, depois de corrigir o YAML em um novo commit, normalmente você deve iniciar uma **nova execução** para validar a correção; reexecutar a antiga não a transforma automaticamente em uma execução do novo commit. Há opções para reexecutar tudo, apenas jobs com falha ou um job específico. Reexecutar workflows e jobs.

3.5 Operar pela CLI e pela API

Comandos de leitura executados em um clone autenticado com GitHub CLI:

```
gh workflow list
gh run list --workflow ci.yaml --limit 10
```

```
gh run view RUN_ID
gh run view RUN_ID --log-failed
gh run download RUN_ID --name relatorio
gh api repos/ORG/REPO/actions/runs/RUN_ID/artifacts
```

Comandos que iniciam ou alteram execuções:

```
gh workflow run ci.yml --ref main
gh run rerun RUN_ID --failed
gh run cancel RUN_ID
gh workflow disable ci.yml
gh workflow enable ci.yml
```

O token da CLI precisa ter acesso ao repositório e às operações solicitadas. Para um repositório privado, ter uma URL de artefato não concede acesso ao arquivo. O download pela interface exige autenticação e acesso de leitura ao repositório. Download de artefatos, API de artefatos.

3.6 Desabilitar, excluir arquivo e excluir execução

Operação	Efeito
Desabilitar workflow	Suspende novos disparos; pode ser revertido
Excluir o arquivo YAML	Remove a definição daquela versão do repositório
Excluir uma execução	Remove um registro específico e os dados associados
Excluir um artefato	Remove o arquivo armazenado daquela execução

Não trate essas operações como sinônimos. A documentação permite excluir uma execução concluída ou com mais de duas semanas e exige acesso de escrita para a operação descrita na interface. Excluir uma execução.

3.7 Escolher a forma de reutilização

Mecanismo	Unidade	Local de consumo	Como chegam atualizações
Template inicial	Arquivo copiado	Criação de um workflow	A cópia precisa ser atualizada
Workflow reutilizável	Um ou mais jobs	<code>jobs.<id>.uses</code>	Conforme a referência escolhida pelo chamador
Ação composta	Sequência de steps	<code>steps[*].uses</code>	Conforme a versão da ação
Âncora YAML	Valor dentro do documento	Alias no mesmo arquivo	Ao editar a definição local

Se a automação precisa escolher runners e coordenar jobs, use um workflow reutilizável. Se precisa se encaixar entre steps existentes, uma ação composta costuma ser a unidade apropriada. Um template pode chamar um workflow reutilizável: os mecanismos podem trabalhar juntos. Conceitos de reutilização.

3.8 Contrato de um workflow reutilizável

Arquivo `.github/workflows/reutilizavel.yml`:

```
name: Produzir uma identificação

on:
```

```

workflow_call:
  inputs:
    prefixo:
      description: Prefixo da identificação
      type: string
      required: true
  outputs:
    identificacao:
      description: Identificação produzida
      value: ${ jobs.gerar.outputs.identificacao }

permissions: {}

jobs:
  gerar:
    runs-on: ubuntu-24.04
    outputs:
      identificacao: ${ steps.meta.outputs.identificacao }
    steps:
      - id: meta
        env:
          PREFIXO: ${ inputs.prefixo }
        run: |
          [[ "$PREFIXO" =~ ^[a-z0-9-]+$ ]] || exit 1
          printf 'identificacao=%s-%s\n' "$PREFIXO" "$GITHUB_RUN_NUMBER" >> "$GITHUB_OUTPUT"

```

Arquivo chamador `.github/workflows/chamador.yml`:

```

name: Consumir identificação
on: workflow_dispatch
permissions: {}

jobs:
  preparar:
    uses: ../github/workflows/reutilizavel.yml
    with:
      prefixo: estudo

  consumir:
    needs: preparar
    runs-on: ubuntu-24.04
    steps:
      - env:
          IDENTIFICACAO: ${ needs.preparar.outputs.identificacao }
        run: printf '%s\n' "$IDENTIFICACAO"

```

O valor percorre quatro interfaces: output do step, output do job, output do workflow reutilizável e contexto `needs` do chamador. A chamada local usa o mesmo commit do chamador. Uma chamada a outro repositório inclui proprietário, repositório, caminho do YAML e `@ref`. Como reutilizar workflows.

Inputs de `workflow_call` são `boolean`, `number` ou `string`. Para segredos, o chamado declara `on.workflow_call - secrets` e o chamador mapeia `jobs.<id>.secrets`. `secrets: inherit` pode ser usado nos cenários suportados da mesma organização ou empresa; não torna segredos acessíveis indiscriminadamente nem os repassa automaticamente por todos os níveis de uma cadeia.

Variáveis `env` do chamador não se propagam como uma interface implícita. Prefira inputs e outputs. Permissões do `GITHUB_TOKEN` podem ser mantidas ou reduzidas no encadeamento, sem elevar o privilégio concedido pelo chamador. Compartilhamento privado exige políticas de acesso compatíveis. Limitações e acesso.

3.9 Decisão rápida do domínio

Situação	Próximo passo
Preciso testar a correção que acabei de enviar	Iniciar uma execução do novo commit
Uma variante falhou por indisponibilidade transitória	Avaliar reexecução seletiva e preservar a evidência
Quero um ponto de partida que cada equipe adapte	Template
Quero centralizar a implementação de vários jobs	Workflow reutilizável
Quero executar três comandos entre steps existentes	Ação composta
Um job está omitido	Examinar condições e dependências antes de investigar o runner

Domínio 3 — Criar e manter ações

4.1 Escolher o tipo de ação

Tipo	Execução	Escolha quando	Ponto de atenção
Java-Script	Runtime Node declarado nos metadados	Precisa de lógica programática, API e portabilidade	Distribuir código executável e dependências
Docker	Contêiner Linux	Precisa controlar ferramentas e bibliotecas do ambiente	Imagem, tempo de inicialização e compatibilidade Linux
Composta	Steps no ambiente do job chamador	Quer reutilizar comandos e outras ações	Shell e ferramentas precisam existir no runner

Uma ação composta não ganha portabilidade apenas por usar YAML. Um script Bash que chama `apt-get` continua dependendo de um ambiente compatível. Uma ação JavaScript que invoca um binário externo também precisa que esse binário exista.

4.2 Estrutura de metadados

O arquivo `action.yml` ou `action.yaml` apresenta o contrato da ação. Campos fundamentais: `name`, `description`, `inputs`, `outputs` e `runs`. A configuração de `runs` depende do tipo:

Tipo	Configuração de <code>runs</code>
JavaScript	<code>using: node24</code> e <code>main: caminho-do-codigo</code>
Docker	<code>using: docker</code> , <code>image</code> e, quando necessário, <code>args</code>
Composta	<code>using: composite</code> e <code>steps</code>

A chave que identifica o tipo da ação é `runs.using`. Não existe `runs.type`, e `runs.steps` pertence apenas à ação composta — usar `steps` em uma ação JavaScript, no lugar de `main`, é erro de contrato.

`required: true` documenta a entrada obrigatória, mas a implementação deve validar ausência, formato e faixa de valores. Em um workflow acionado por `workflow_dispatch`, uma entrada obrigatória faz o GitHub **pedir o valor à pessoa** antes de iniciar a execução; não há execução silenciosa nem erro imediato, e nada é preenchido a partir da execução anterior. Em ações compostas, declare `shell` nos steps com `run` e mapeie outputs com `value`. Sintaxe de metadados.

A chave `outputs` é o mecanismo pelo qual uma ação entrega dados aos steps seguintes. Dentro da própria ação, o valor de um output referencia o step produtor pela forma completa `${{ steps.<id-do-step>.outputs.<nome> }}` — o `id` do step é obrigatório nessa referência.

O GitHub converte cada entrada em variável de ambiente segundo uma convenção fixa: prefixo `INPUT_`, nome em **maiúsculas** e espaços substituídos por `_`. Uma entrada `numOctocats` vira `INPUT_NUMOCTOCATS`; `my input name` vira `INPUT_MY_INPUT_NAME`.

4.3 Ação composta com entrada e saída

Arquivo `.github/actions/identificacao/action.yml`:

```

name: Gerar identificação de estudo
description: Valida um prefixo e devolve uma identificação da execução

inputs:
  prefixo:
    description: Letras minúsculas, números e hífens
    required: true

outputs:
  valor:
    description: Prefixo seguido do número da execução
    value: ${{ steps.gerar.outputs.valor }}

runs:
  using: composite
  steps:
    - id: gerar
      shell: bash
      env:
        PREFIX0: ${{ inputs.prefixo }}
      run: |
        if [[ ! "$PREFIX0" =~ ^[a-z0-9-]+$ ]]; then
          printf '::error::Prefixo inválido.\n'
          exit 1
        fi
        printf 'valor=%s-%s\n' "$PREFIX0" "$GITHUB_RUN_NUMBER" >> "$GITHUB_OUTPUT"

```

Arquivo `.github/workflows/testar-acao.yml`:

```

name: Testar ação local
on: workflow_dispatch
permissions:
  contents: read

jobs:
  teste:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7
        with:
          persist-credentials: false
      - id: identificacao
        uses: ../github/actions/identificacao
        with:
          prefixo: gh200
      - env:
          RESULTADO: ${{ steps.identificacao.outputs.valor }}
        run: printf '%s\n' "$RESULTADO"

```

O checkout disponibiliza a ação local. O `uses` aponta para o **diretório** que contém seus metadados, não diretamente para `action.yml`. A entrada é passada ao shell por `env`, validada e escrita como output. Não há acesso implícito ao contexto `secrets` dentro da ação composta; o chamador fornece valores necessários explicitamente por entradas ou ambiente. Criar uma ação composta.

Exercício: execute com `prefixo: gh200`, depois com um valor contendo espaço. A primeira chamada deve produzir uma identificação; a segunda deve reprovar com uma mensagem clara. Explique por que o output do step interno precisa ser mapeado no output da ação.

4.4 Ação JavaScript mínima

Crie um diretório `.github/actions/validar-nome/` com dois arquivos. Este exemplo usa apenas a biblioteca padrão do Node.

Arquivo `.github/actions/validar-nome/action.yml`:

```
name: Validar nome de recurso
description: Valida um nome curto e produz sua versão em letras maiúsculas
inputs:
  nome:
    description: Nome com letras, números e hífens
    required: true
outputs:
  normalizado:
    description: Nome validado em letras maiúsculas
runs:
  using: node24
  main: index.cjs
```

Arquivo `.github/actions/validar-nome/index.cjs`:

```
const fs = require('node:fs');

try {
  const nome = process.env.INPUT_NOME || '';
  if (!/^[a-zA-Z0-9-]{1,40}$/.test(nome)) {
    throw new Error('O nome deve ter de 1 a 40 letras, números ou hífens.');
```

A extensão `.cjs` torna explícito o uso de CommonJS. A validação impede que quebras de linha fornecidas pelo usuário alterem o formato do arquivo de outputs. O runtime `node24` executa a ação; ele não seleciona, por si só, a versão do Node usada pela aplicação nos demais steps.

Em ações maiores, o Actions Toolkit oferece funções como `getInput`, `setOutput`, `setFailed` e clientes para APIs. As dependências precisam estar disponíveis na distribuição, frequentemente por um bundle versionado em `dist/`. Publicar somente TypeScript ou um `package.json` não garante uma ação executável: o consumidor não instala suas dependências automaticamente. Criar uma ação JavaScript.

4.5 Ações Docker

Uma ação Docker reúne metadados, imagem e ponto de entrada. O diretório pode conter `action.yml`, `Dockerfile`, um script de entrada e os recursos necessários.

Três detalhes operacionais derrubam esse tipo de ação com frequência:

- **Só roda em Linux.** Um runner auto-hospedado precisa de sistema operacional Linux com Docker instalado. Não adianta instalar Docker em runner Windows.
- **O nome do arquivo diferencia maiúsculas de minúsculas.** É `Dockerfile` — D maiúsculo, f minúsculo. Um `dockerfile` no repositório faz a ação não executar.
- **Entradas chegam ao contêiner por `args`.** As entradas declaradas em `inputs` não ficam automaticamente disponíveis dentro do contêiner: é a chave `args` no arquivo de metadados que as repassa. A sintaxe `process.env.INPUT_*` pertence a ações JavaScript, não a esse caminho.

Para conferir o que o contêiner realmente recebeu, execute `env` dentro dele e leia a saída no log. Isso vale quando a ação é sua; em uma ação Docker de terceiros você não controla o ponto de entrada e precisa recorrer aos logs da execução. O comando `docker logs` não alcança a execução do Actions, e `git log` não tem relação com o problema.

Trecho ilustrativo de metadados:

```
name: Analisar um arquivo
description: Executa uma ferramenta empacotada em contêiner
inputs:
  arquivo:
    description: Caminho do arquivo no workspace
    required: true
runs:
  using: docker
  image: Dockerfile
  args:
    - ${inputs.arquivo}
```

Esse trecho depende de um `Dockerfile` e da implementação da ferramenta. No ponto de entrada, receba argumentos como dados, preserve o código de saída e valide os caminhos. Não construa um comando com `eval` a partir do argumento.

Falha	O que conferir
Build da imagem falha	Caminhos do contexto, instruções e arquivos copiados
Permission denied	Permissão de execução do script e usuário da imagem
Exec format error	Arquitetura e formato do executável
Script existe, mas não executa	Shebang, interpretador e finais de linha
Arquivo não encontrado	Diretório de trabalho e workspace montado
Ação não roda em macOS/Windows	Ação Docker exige runner Linux com Docker

O contêiner melhora a consistência das dependências, mas não elimina confiança no código da ação nem no conteúdo da imagem. As exigências do host constam na referência de runners.

4.6 Testar e distribuir

Um contrato de ação deve explicar entradas, saídas, permissões, sistemas suportados e comportamento em falhas. Para o exemplo de validação, os casos relevantes são entrada válida, entrada ausente, caracteres proibidos e tamanho excessivo.

Forma de distribuição	Característica
Diretório no mesmo repositório	A versão acompanha o commit do consumidor
Repositório público	Pode ser referenciado se as políticas permitirem

Forma de distribuição	Característica
Repositório privado ou interno	Requer configuração de acesso compatível
Marketplace	Adiciona descoberta e apresentação pública

Estar no Marketplace não significa que o código foi revisado e aprovado pelo GitHub. A documentação atual exige repositório público, um arquivo de metadados na raiz e nome que atenda às regras de unicidade; ações em subdiretórios podem existir, mas não são listadas automaticamente. Também é necessário aceitar os termos aplicáveis. Publicação no Marketplace.

Requisitos para a publicação sair imediatamente, sem revisão manual:

- Repositório **público**.
- **Uma única** ação por repositório, sem arquivos de workflow no repositório.
- `action.yml` ou `action.yaml` no **diretório raiz**.
- `name` único no Marketplace, que não coincida com uma ação já publicada, com um recurso do GitHub, com uma categoria existente nem com o nome de usuário ou organização de terceiros — só a própria organização pode publicar uma ação chamada `github`.
- Autenticação de dois fatores ativa para publicar a release.

O fluxo de publicação parte do arquivo de metadados no repositório: um banner oferece **Draft a release**; em "Release Action", marque **Publish this Action to the GitHub Marketplace**; escolha a **Primary Category** (e, opcionalmente, uma secundária) para tornar a ação localizável; informe a tag de versão e o título da release; publique.

Se a caixa **Publish** aparecer desabilitada, o motivo é que a conta dona do repositório ainda não aceitou o **GitHub Marketplace Developer Agreement** — não é problema de metadados nem de workflow. Aceite o acordo pelo link exibido ou peça ao owner da organização que o faça.

Ao decidir usar uma ação de terceiros, avalie o `action.yml` para confirmar o que ela faz, verifique a presença no Marketplace e a marca de **verified creator**, que indica que o GitHub verificou o criador como organização parceira. Essa marca fala do **criador**, não garante auditoria do código e não substitui sua revisão. Quantidade de estrelas mede popularidade, não integridade.

Um `README.md` completo é parte do contrato da ação: descrição detalhada do que ela faz, argumentos de entrada e saída obrigatórios e opcionais, segredos utilizados e variáveis de ambiente utilizadas. Tutoriais interativos e guias de solução de problemas podem existir, mas como material complementar — o README existe para permitir o uso.

4.7 Versionamento, releases e imutabilidade

Referência	Propriedade	Uso consciente
@main	Branch mutável	Acompanhar desenvolvimento com risco de alterações
@v1	Tag normalmente atualizada pelo mantenedor	Receber versões compatíveis conforme a política do projeto
@v1.2.3	Tag de versão	Legível, mas uma tag comum não é imutável por definição
@SHA_COMPLETO	Commit específico	Revisar e fixar o código exato
Release imutável	Proteção de uma release e da tag associada	Publicação que não pode ser alterada como uma release comum

Ao habilitar releases imutáveis, use uma tag específica para a release e mantenha eventuais tags móveis de compatibilidade sem vinculá-las a releases imutáveis. Não planeje mover uma tag que se tornou protegida por esse mecanismo. Releases imutáveis e tags.

A convenção de numeração é o **versionamento semântico**, no formato `MAJOR.MINOR.PATCH`. Ele comunica o impacto da mudança: incompatibilidade no major, funcionalidade no minor, correção no patch. Nomes arbitrários ou baseados em data e hora não transmitem essa informação, e hashes de commit identificam código, mas não descrevem compatibilidade.

Boas práticas de release que caem em prova:

- Criar e validar a release em uma **branch de release** (como `release/v1`) antes de criar a tag (`v1.0.2`) — não diretamente na branch padrão.
- Mover a tag de major (`v1`, `v2`) para apontar ao commit da release atual, de modo que quem referencia `@v1` receba correções compatíveis.
- Abrir uma **nova tag major** quando a mudança quebra workflows existentes. Quem ainda referencia `v1` continua funcionando; quem quiser migrar passa a referenciar `v2`.
- Publicar versões major com sufixo beta (`v2-beta`) enquanto o status for experimental, removendo o sufixo quando a release amadurecer.

Do lado de quem consome, a recomendação é referenciar a **versão major** e descer para uma versão mais específica apenas se surgir um problema. Referenciar a branch padrão expõe o workflow a código possivelmente instável; atualizar automaticamente para a última versão introduz mudanças não avaliadas.

Uma estratégia de release automatizada evita commitar dependências na branch padrão e as inclui apenas nos commits de release taguados, fazendo o build durante a release. Isso reforça a referência por tag nomeada ou SHA em vez da branch de desenvolvimento.

Se você adotar o SHA como forma de versionamento, use o **SHA completo**. O Git reconhece formas abreviadas, mas a abreviação não oferece a mesma garantia para gestão de releases.

O programa do exame também cita ações imutáveis e fontes de registro. Ao analisar uma proposta de distribuição, identifique **origem**, **referência resolvida**, **proteção contra alterações** e **compatibilidade do runner**. Git refs, imagens `docker://` e releases não têm contratos idênticos. Uma tag com aparência de versão ou uma imagem hospedada em um registry não implica imutabilidade. Use a documentação da funcionalidade citada no cenário; este material não pressupõe uma sintaxe genérica de registry para todos os tipos de ação.

4.8 Decisão rápida do domínio

Situação	Escolha
Reaproveitar uma sequência de comandos	Ação composta

Situação	Escolha
Manipular APIs e dados em várias plataformas	JavaScript com dependências distribuídas
Empacotar uma ferramenta Linux e suas bibliotecas	Ação Docker
Fixar exatamente a versão revisada	SHA completo verificado no repositório de origem
Receber atualizações	Processo de atualização e testes; fixação não atualiza sozinha
Validar uma ação antes de publicar	Testar a mesma distribuição que o consumidor executará

Domínio 4 — Gerenciar o GitHub

Actions para a organização

5.1 Governança em camadas

Uma empresa pode estabelecer regras superiores; a organização define padrões dentro delas; o repositório aplica suas configurações permitidas. Ao investigar uma restrição, procure a origem da política. Editar o workflow não amplia uma permissão que o administrador bloqueou.

Controle	Pergunta que resolve
Ativação de Actions	Quais repositórios podem executar automações?
Política de ações e workflows permitidos	De quais origens é permitido executar código?
Política de token	Que permissões padrão a automação recebe?
Compartilhamento de componente privado	Quais consumidores podem acessar esse componente?
Runner groups	Quais repositórios ou workflows podem usar determinados runners?
Ambientes protegidos	Quais condições devem ser atendidas para implantar?
Retenção e orçamento	Quanto tempo guardar dados e quanto consumir?

Se uma política permite apenas ações da própria organização, ações do GitHub, como `actions/checkout`, também precisam estar contempladas pela configuração adequada. Não confunda “publicada pelo GitHub” com “pertence à minha organização”. Configurações de Actions do repositório.

5.2 Templates organizacionais

Estrutura de exemplo no repositório `.github` de uma organização:

```
.github/  
├── workflow-templates/  
│   ├── verificar.yml  
│   └── verificar.properties.json
```

Nesse desenho, `.github` é o **nome do repositório**, e `workflow-templates` fica em sua raiz. Isso difere do diretório `.github/workflows/` dentro de um repositório de aplicação.

Quem tem acesso de escrita ao repositório `.github` da organização cria esses templates, que ficam disponíveis para os demais membros criarem novos workflows nos repositórios públicos e privados da organização. Do lado de quem consome, a página "Choose a workflow" lista os templates recomendados e o botão para adotar um deles é **Configurar** — não "Use", "Install" nem "Deploy". Se o template referencia um segredo, como `${{ secrets.token }}`, o repositório precisa ter esse segredo criado com o mesmo nome antes do primeiro uso; remover a referência quebra o workflow e substituí-la pelo valor em texto claro anula a proteção.

Templates organizacionais existem para **economizar tempo, promover consistência e difundir boas práticas** entre equipes. Some a isso um padrão documentado de nomenclatura e organização, aplicado em toda a organização, que identifique tipo, finalidade e versão de cada componente. Deixar cada equipe inventar o próprio padrão gera inconsistência quando os componentes começam a circular; abreviações arbitrárias dificultam identificar o que cada arquivo faz. Registre em uma wiki ou em um markdown acessível a todos: repositórios de armazenamento, convenções de

nomes de arquivos e pastas, localização dos componentes compartilhados, planos de manutenção contínua e diretrizes de contribuição. Instruções de configuração de workflows individuais não entram nessa lista — elas duplicam o que os próprios padrões já definem.

Arquivo `workflow-templates/verificar.properties.json`:

```
{
  "name": "Verificação de projeto",
  "description": "Ponto de partida para a integração contínua da equipe",
  "categories": ["JavaScript"],
  "filePatterns": ["package.json$"]
}
```

Um template público pode atender repositórios de diferentes visibilidades. Templates em repositório `.github` interno ou privado têm alcance mais restrito e exigem acesso apropriado. A cópia criada não recebe atualizações automáticas do template. Templates não públicos.

5.3 Runners hospedados e auto-hospedados

Aspecto	Hospedado pelo GitHub	Auto-hospedado
Provisionamento	Gerenciado pela plataforma	Responsabilidade da organização
Sistema e ferramentas	Imagens publicadas pelo provedor	Imagem e instalações administradas pela equipe
Rede	Recursos oferecidos conforme tipo e plano	Integração com a rede escolhida pela equipe
Limpeza	Ambiente gerenciado por job nos runners convencionais	Precisa ser projetada e verificada
Capacidade	Limites e opções da plataforma	Dimensionamento, filas e disponibilidade próprios
Custos	Consumo e condições do plano	Infraestrutura, operação e condições aplicáveis

Escolha runner auto-hospedado quando há uma necessidade concreta de ambiente, rede ou hardware. Inclua atualização, isolamento e descarte no desenho. Uma máquina que executa código de repositórios diferentes pode carregar resíduos e credenciais de uma execução para outra se a operação não controlar esse risco. Conceito de runner auto-hospedado.

5.4 Labels e grupos

Labels descrevem características usadas no roteamento. Grupos controlam o acesso ao conjunto de runners. Ter o label correto não concede acesso a um grupo restrito.

Trecho de seleção, dependente de um grupo e labels previamente configurados:

```
runs-on:
  group: implantacao-interna
  labels: linux-x64-deploy
```

Use grupos para separar fronteiras de confiança: testes de contribuição externa, builds internos e implantação de produção não precisam compartilhar a mesma capacidade. Restrições a workflows selecionados podem tornar o acesso mais específico que uma lista de repositórios, conforme o recurso disponível. Gerenciar grupos de runners.

Duas regras práticas decidem questões sobre essa dupla:

- **Labels são cumulativas.** O runner precisa ter **todas** as labels exigidas pelo job para ser elegível. Não basta coincidir com uma delas, e labels não são atribuídas automaticamente a partir das características da máquina — você as

define.

- **Runner novo entra no grupo padrão.** Um runner recém-registrado é atribuído automaticamente ao grupo *Default*. Se a equipe não o enxerga no grupo dela, a causa mais provável é essa, e a correção é movê-lo — antes de investigar permissões, rede ou inicialização.

Quando o critério é controle de custo — por exemplo, um modelo de *chargeback* em que a equipe não quer que workflows de desenvolvimento consumam os runners que ela paga — a resposta é **grupo**, porque a questão é acesso.

Quando o critério é escolher a máquina adequada ao trabalho, a resposta é **label**.

5.5 Operação, rede e imagens

O runner auto-hospedado inicia conexões HTTPS de saída com o GitHub, normalmente na porta 443. Também precisa alcançar os serviços usados pelos jobs: registries, armazenamento de artefatos e fontes de dependências. Não basta liberar apenas a página principal do GitHub. Preserve logs fora de runners efêmeros para conseguir investigar depois do descarte. Requisitos de comunicação e operação.

A conexão é **sempre de saída**: o runner abre um *long poll* HTTPS para o GitHub e aguarda atribuição de job. Por isso você **não** precisa liberar conexões de entrada do GitHub para o runner. Também não é exigida internet de alta velocidade nem acesso a todas as APIs públicas — o necessário é conectividade estável com os hosts documentados.

Quando a rede exige proxy, configure a variável de ambiente `https_proxy` na máquina do runner (com `http_proxy` e `no_proxy` conforme o caso), incluindo credenciais de autenticação básica se o proxy pedir. Nomes como `proxy_server`, `network_proxy` ou `outbound` não existem. O proxy é suportado, mas não é requisito.

Para validar se o runner alcança tudo de que precisa, execute a aplicação do runner com o parâmetro `--check`, informando a URL e o token de autenticação. Ele testa a conectividade com os serviços exigidos. Não existem `--diag`, `--validate-network` nem `--verify-connection`.

Na interface, cada runner registrado aparece com nome, labels e um **status**, que é um de três valores:

Status	Significado
idle	Conectado ao GitHub e pronto para executar jobs
active	Executando um job neste momento
offline	Sem conexão — máquina desligada, aplicação parada ou comunicação bloqueada

Não existe status `overloaded`.

Runners efêmeros em contêiner se atualizam sozinhos sempre que sai uma versão nova do software do runner, o que gera interrupções repetidas. Desligar a atualização automática devolve o controle: você atualiza a versão na imagem do contêiner no seu próprio cronograma.

Para diagnosticar a capacidade, acompanhe tempo em fila, ocupação, duração dos jobs, falhas de inicialização e versão do software do runner. Se o job exige uma combinação de labels para a qual não existe runner acessível, aumentar o timeout do step não resolve.

Sobre **allowlist de IP** com runners hospedados: o GitHub publica as faixas de endereços pela API, atualizadas semanalmente. Como runners Windows e Ubuntu ficam no Azure, as faixas são amplas e mudam com frequência — manter a allowlist sincronizada toda semana é trabalhoso e propenso a erro. Liberar toda a faixa do Azure resolve a operação e destrói o propósito da restrição. *Larger runners* com faixa de IP estática são a saída intermediária: poucas entradas, estáveis, sem a sobrecarga de operar runners próprios.

`ubuntu-latest` e `windows-latest` são aliases de imagens mantidas, não congelamentos do sistema. Mesmo um label como `ubuntu-24.04` recebe atualizações de ferramentas. Consulte o log de preparação do job e as versões publicadas das imagens; configure runtimes com ações `setup-*` quando precisar de uma versão determinada. O toolca-

che guarda ferramentas disponíveis localmente e não é o mesmo serviço de cache de dependências. Imagens oficiais de runners.

5.6 Segredos, variáveis e escopo

Tipo	Contexto	Exemplos	Tratamento
Segredo	<code>secrets</code>	Token externo, senha, chave privada	Proteção e acesso controlado
Variável de configuração	<code>vars</code>	Região, nome de recurso, opção de build	Configuração não sensível
Variável de ambiente do YAML	<code>env</code>	Opção para comandos do job	Escopo do workflow, job ou step

Uma variável não deve ser usada para esconder credenciais. Segredos e variáveis podem existir em organização, repositório e ambiente, mas têm disponibilidade e políticas próprias.

Para segredos de mesmo nome, o escopo mais específico tem prioridade: ambiente, repositório e organização. Segredos de organização precisam permitir o repositório consumidor; os de ambiente exigem que o job use aquele ambiente e satisfaça suas proteções. Referência de segredos.

Os três escopos possíveis são **organização, repositório e ambiente**. Não existe segredo no escopo de workflow.

Um segredo criado dentro de um repositório só funciona naquele repositório — não há como estendê-lo a outros por política, porque a política de acesso pertence ao segredo de organização. Quando várias equipes precisam da mesma credencial, crie o segredo na **organização** e configure a política que lista os repositórios autorizados. Isso mantém uma cópia única, faz a rotação valer para todos de uma vez e dá controle granular de acesso. Ao decidir entre organização e ambiente, pesem quantos repositórios precisam da chave, com que frequência ela é rotacionada e que controle de acesso cada equipe exige; exigência de aprovação individual antes do uso é característica de **ambiente**, não de organização.

Em repositório de conta pessoal, só o dono cria segredos. Se você colabora no repositório de outra pessoa e não consegue adicionar um segredo, o caminho é pedir ao dono — não abrir PR com o valor, nem fazer fork, nem acionar o suporte.

O limite de tamanho de um segredo é 48 KB. Acima disso, o procedimento documentado é cifrar o arquivo com **GPG**, versionar o arquivo cifrado no repositório e guardar apenas a *passphrase* de decifragem como segredo. Registrar o valor em **Base64 não é criptografia** e não protege nada — é apenas codificação.

O GitHub **oculta automaticamente** segredos impressos nos logs, substituindo-os por asteriscos. Isso mitiga vazamentos acidentais, mas não autoriza imprimir segredos de propósito, e a ocultação depende do valor coincidir exatamente com o segredo registrado.

Para variáveis de configuração, a documentação também descreve precedência por escopo, com uma nuance: **variáveis de ambiente ficam disponíveis no runner depois que o job começa e não sobrescrevem valores já estabelecidos nos contextos** `env` e `vars`. Evite usar uma regra de precedência simplificada para prever valores em campos avaliados antes dessa fase. Para `env` declarado no YAML, prevalece o nível mais específico: step, job e workflow. Referência de variáveis.

5.7 Uso de segredos

Trecho de step, em um job autorizado a acessar o segredo:

```
- name: Verificar se a credencial foi disponibilizada
  env:
    API_TOKEN: ${ secrets.API_TOKEN }
  run: |
```

```
if [ -z "$API_TOKEN" ]; then
    printf '::error::Credencial não disponível neste contexto.\n'
    exit 1
fi
printf 'Credencial disponível; valor não exibido.\n'
```

Um segredo ausente resulta em valor vazio na expressão. Forks e eventos do Dependabot têm restrições próprias. Não presuma que um segredo existente nas configurações será entregue a todo evento. Também não teste um segredo diretamente em qualquer `if`: use o padrão documentado com variável de ambiente no escopo permitido. Como usar segredos.

Mascaramento de log não é autorização de acesso e não é uma garantia contra toda transformação do valor. Evite registrar credenciais, incluí-las em artefatos ou imprimi-las em estruturas JSON completas. Um output é uma interface de dados, não um substituto para um cofre de segredos.

Evite passar segredos entre processos pela **linha de comando**. Argumentos de linha de comando podem ficar visíveis a outros usuários da máquina, aparecer em histórico de shell e ser capturados por eventos de auditoria. Prefira variáveis de ambiente no escopo do step. Se não houver alternativa, aplique as regras de aspas corretas do shell.

A sintaxe de referência é sempre `${{ secrets.NOME }}`, tanto em `env:` quanto em `with:`. Não existe `secrets.environment.NOME`, e escrever apenas `${{ NOME }}` não resolve nenhum segredo. Também não existe pacote `actions/secrets` para decifrar valores: o GitHub decifra automaticamente e entrega pelo contexto `secrets`.

5.8 Administração por CLI e REST

Exemplos de leitura:

```
gh secret list --repo ORG/REPO
gh variable list --repo ORG/REPO
gh api repos/ORG/REPO/actions/permissions
gh api repos/ORG/REPO/actions/variables
```

Exemplos de gravação de configuração de laboratório:

```
gh variable set REGIAO --body 'us-east-1' --repo ORG/REPO
gh secret set API_TOKEN --repo ORG/REPO
```

O segundo comando permite fornecer o segredo interativamente, sem incluí-lo na linha de comando. Na REST API de segredos, o cliente consulta a chave pública do escopo e envia o valor criptografado; não envia simplesmente um campo com a senha em texto claro. Endpoints de listagem devolvem metadados, não o conteúdo original dos segredos. API de segredos, API de variáveis.

Políticas também têm endpoints específicos. Escolha o nível correto — repositório, organização ou empresa — e a identidade autorizada para administrá-lo. Ter `contents: write` em um job não equivale a permissão administrativa sobre todas as configurações de Actions. API de permissões.

5.9 Exemplo de desenho organizacional

Considere `loja-api` e `loja-web`, cada um com ambientes `teste` e `producao`.

Item	Local proposto	Motivo
Workflow comum de validação	Repositório de automação compartilhado	Manter uma implementação reutilizável

Item	Local proposto	Motivo
Variável com domínio comum de homologação	Organização, para os repositórios selecionados	Reduzir duplicação de configuração
Segredo exclusivo de uma aplicação	Repositório ou ambiente específico	Restringir o alcance
Papel de nuvem para produção	Federação OIDC vinculada ao ambiente	Credencial temporária com confiança restrita
Aprovação de implantação	Ambiente <code>producao</code>	Avaliação antes do acesso à implantação
Runners com acesso à rede de produção	Grupo restrito	Limitar quem pode alcançar a rede
Ações externas aceitas	Política organizacional	Controlar dependências executáveis

Exercício: explique por que permitir o repositório no runner group não substitui a proteção do ambiente. Depois descreva quem pode alterar o workflow que usa ambos. A governança precisa considerar quem edita a automação, não apenas quem aperta o botão de execução.

5.10 Decisão rápida do domínio

Situação	Recurso
Configuração não sensível para vários repositórios	Variável de organização com acesso selecionado
Credencial específica de produção	Segredo de ambiente ou federação com escopo equivalente
Restringir acesso a runners de uma rede	Runner group
Escolher máquina por capacidade	Labels
Fornecer um YAML inicial à organização	<code>workflow-templates/</code> na raiz do repositório chamado <code>.github</code>
Bloquear uma origem de código executável	Política de ações e workflows permitidos

Domínio 5 — Automação segura e otimizada

6.1 Identidade, permissão e confiança

Avalie cada job usando quatro perguntas: **qual código será executado, quem pode alterá-lo, quais credenciais serão entregues e quais sistemas o runner alcança?** Um workflow pode ter poucos steps e ainda representar grande risco se executar contribuições externas com acesso de produção.

Credencial	Uso adequado	Limitação a considerar
GITHUB_TOKEN	Operações permitidas no repositório do workflow	Escopo e duração ligados ao job
Token de instalação de GitHub App	Automação com identidade própria e acesso selecionado	Instalação e permissões precisam ser administradas
PAT	Integração que exige identidade e acesso do usuário	Ciclo de vida, expiração e dependência da conta
OIDC	Federação com provedor externo	Exige confiança configurada no provedor e permissão de solicitar identidade

Escolha uma credencial pelo alcance necessário. Substituir qualquer erro de permissão por um PAT amplo cria acesso excessivo e pode ocultar o problema original.

6.2 Permissões do GITHUB_TOKEN

O GitHub cria um token para cada job, limitado ao repositório que contém o workflow. Ele expira com o job ou conforme seu tempo de vida efetivo. Na documentação atual, jobs hospedados pelo GitHub têm duração máxima de seis horas; em runners próprios, a autenticação por esse token não se estende além de 24 horas, mesmo que o job tenha limite maior. GITHUB_TOKEN.

Declare permissões mínimas no workflow e amplie apenas no job que precisa delas. Quando uma configuração explicita determinadas permissões, as não especificadas ficam como `none`, ressaltados os comportamentos documentados da plataforma.

Trecho de permissões para um job que gera atestados:

```
permissions:
  contents: read
  id-token: write
  attestations: write
```

`id-token: write` autoriza solicitar um token OIDC. Não concede `contents: write`, permissão de publicar pacotes nem acesso automático a recursos da nuvem. Para GitHub Packages, avalie separadamente a necessidade de `packages: write` e o acesso ao pacote.

Eventos produzidos pelo GITHUB_TOKEN têm regras para evitar recursão. `workflow_dispatch` e `repository_dispatch` são exceções; a documentação atual também descreve certos eventos de PR criado ou atualizado por esse token iniciados em estado que exige aprovação. Não memorize “esse token nunca dispara workflows” como uma regra absoluta. Comportamento dos eventos do token.

6.3 Entradas não confiáveis e injeção

Títulos de PR, nomes de branches, texto de issues e inputs podem ser controlados por outra pessoa. Interpolar esses valores diretamente no corpo de `run` mistura dados e código.

Padrão a evitar, mostrado como texto para análise:

```
run: echo "${{ github.event.pull_request.title }}"
```

Trecho de step que mantém o valor como dado:

```
- name: Registrar o título como texto
  env:
    TITULO_PR: "${{ github.event.pull_request.title }}"
  run: printf '%s\n' "$TITULO_PR"
```

A variável e as aspas impedem que o shell reinterprete o conteúdo como parte do script nessa operação. Para comandos com significado especial, valide também o formato e os argumentos; uma entrada segura para imprimir pode ser inadequada como caminho ou opção de ferramenta. Uso seguro de Actions.

6.4 Pull requests e fronteiras de privilégio

`pull_request_target` atende automações no contexto da base, como gestão de metadados. Não use esse privilégio para baixar e executar código não confiável do PR com acesso a segredos ou rede interna. Atenção também a `workflow_run`: receber um artefato de uma execução sem privilégios não torna confiável o conteúdo desse artefato.

No cenário de build e implantação, separe a verificação de contribuição da operação privilegiada e faça esta última validar origem, referência e integridade do que consome. Proteja alterações nos workflows e revise ações externas. O SHA fixa o código selecionado; não prova que esse código é benigno nem congela tudo que ele baixa durante a execução. Orientações de segurança.

6.5 OIDC para credenciais temporárias

O fluxo de federação segue esta sequência:

- 1. O job é autorizado a solicitar uma identidade com `id-token: write`.
- 2. O GitHub emite um JWT com claims sobre a execução.
- 3. O provedor externo valida emissor, audiência e condições de confiança.
- 4. Se a identidade corresponde à política, o provedor entrega credenciais temporárias.
- 5. A operação é limitada pelas permissões do papel ou identidade no provedor.

A confiança precisa restringir o repositório e o contexto adequado, como branch ou ambiente. Uma condição de audiência correta com um `sub` excessivamente amplo ainda pode permitir acesso indesejado.

Atualização relevante: repositórios criados após 15/07/2026 usam, no GitHub.com, um formato de subject com IDs imutáveis de proprietário e repositório. Repositórios anteriores podem manter o formato antigo ou aderir à mudança; renomeações e transferências também merecem atenção. A política de confiança deve corresponder ao formato efetivamente utilizado. Esse comportamento não se aplica da mesma forma ao GHES. Referência OIDC.

Para AWS, a integração usual troca a identidade OIDC por credenciais temporárias de um papel IAM. A audiência usada pela ação oficial é `sts.amazonaws.com`; a política de confiança também deve restringir o subject. Quando o job usa um ambiente, esse contexto participa do subject. OIDC na AWS.

Não há implantação real nos exemplos completos deste guia. Para transformar o desenho em uma implantação, configure primeiro o provedor, o papel, as condições e o ambiente; depois adicione a ação de autenticação e os comandos de publicação específicos da aplicação.

6.6 Ambientes e aprovações

Um job que declara `environment: producao` fica sujeito às regras configuradas nesse ambiente. Elas podem incluir revisores, restrições de branches/tags e outras proteções, conforme o plano e a visibilidade. Um nome de ambiente sem regras não cria aprovação por si só.

A proteção deve ocorrer antes da disponibilização dos recursos sensíveis do ambiente. Avalie também a possibilidade de o próprio autor aprovar, a autoridade para contornar regras e quem pode alterar a configuração. Gerenciar ambientes.

As proteções configuráveis em um ambiente incluem **exigir revisores**, **impedir a auto-aprovação** e **restringir quais branches podem implantar**. Cobertura mínima de código não é proteção de implantação: isso pertence ao CI, verificado por um job ou por check obrigatório.

A prevenção de auto-aprovação responde a um caso específico: quem iniciou a execução não pode aprová-la. Se alguém relata não conseguir aprovar a própria implantação, verifique essa configuração antes de investigar permissões ou plano da conta.

O ciclo de vida de um job que aguarda revisão:

Situação	Resultado
Aguardando revisão	Status <code>Waiting</code> ; nenhum step executa
Aprovado	O job prossegue
Rejeitado	O workflow falha
Sem decisão por 30 dias	O job falha automaticamente

Os jobs que referenciam um ambiente só começam quando **todas** as regras de proteção passam — não algumas.

6.7 Atestados e proveniência

Um checksum permite comparar bytes. Um atestado associa um artefato a informações verificáveis de produção, como identidade do workflow e origem do build. A verificação precisa conferir não apenas a existência de uma assinatura, mas se o produtor e a origem são os esperados.

Atestados não substituem testes, revisão de código ou análise de vulnerabilidades. Eles ajudam a responder **de onde veio esse artefato e como foi produzido?** Conceitos de atestados.

Fluxo de exercício: produza um arquivo, gere um atestado pelo mecanismo oficial, baixe o mesmo arquivo e execute a verificação autenticada:

```
gh attestation verify dist/aplicacao.tar.gz --repo ORG/REPO
```

Esse comando pressupõe um arquivo já atestado por aquele repositório. Disponibilidade, permissões e integração com registries dependem do cenário. A geração e a verificação devem fazer parte do processo, em vez de apenas gerar metadados que ninguém consulta. Gerar e verificar proveniência.

6.8 Otimização de tempo e custo

Considere a soma do trabalho executado, o tempo de espera e o armazenamento. Seis jobs de quatro minutos representam 24 minutos de execução agregada, mesmo que em paralelo a pessoa espere aproximadamente quatro minutos, mais inicialização e filas. Isso não é uma estimativa de preço: tarifa, runner e condições do plano afetam a cobrança.

Problema	Melhoria	Trade-off
Todos os commits repetem verificações obsoletas	Cancelar execução anterior da mesma branch	Não aplicar cegamente a tarefas que não podem ser interrompidas
Matriz cresce sem evidência de valor	Selecionar combinações representativas	Preservar a cobertura necessária
Dependências são baixadas repetidamente	Cache com identidade adequada	Medir taxa de acerto e custo de restauração
Muitos artefatos grandes	Selecionar arquivos e retenção	Manter evidência suficiente para diagnóstico
Reprocessamento após uma falha isolada	Reexecução seletiva	Garantir que a causa foi entendida
Job monopoliza um runner	Timeout e divisão de trabalho	Evitar fragmentação que aumenta overhead
Compressão demora mais que o upload	Ajustar compressão para o conteúdo	Arquivos maiores podem consumir mais armazenamento

Trecho para CI que pode descartar resultados antigos da mesma referência:

```
concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

`concurrency` controla a coexistência no grupo. O comportamento padrão mantém no máximo uma execução pendente no grupo e pode substituir a pendente anterior. A documentação atual também oferece `queue: max` para enfileirar várias pendentes; essa opção não pode ser combinada com `cancel-in-progress: true`. Escolha explicitamente entre descartar trabalho obsoleto e preservar a fila de operações. Controle de concorrência.

6.9 Retenção e administração programática

Retenção é uma decisão de custo e de capacidade de investigação. Use períodos menores para arquivos temporários e prazos compatíveis com as necessidades de auditoria para evidências. A configuração de logs/artefatos não deve ser presumida como idêntica à retenção de todo metadado de execução. A documentação atual descreve a expansão dessas políticas e suas condições de implantação. Retenção organizacional.

Para automatizar limpeza, liste primeiro os artefatos pela API, avalie idade, expiração, origem e necessidade de preservação e só então aplique exclusões previstas pela política. Não existe um único comando YAML que governe toda a retenção da empresa. Configurações e recursos têm endpoints e permissões próprios. API de artefatos.

6.10 Decisão rápida do domínio

Situação	Abordagem
Job só lê código	<code>contents: read</code>
Publicação precisa de privilégio adicional	Conceder no job específico
Autenticação em nuvem com suporte a federação	OIDC com confiança restrita

Situação	Abordagem
Dado externo entra em script	env , aspas e validação de formato/uso
Dependência externa executa código	Revisão da origem, SHA e atualização controlada
Produção exige aprovação	Ambiente com regra efetivamente configurada
Build precisa de prova de origem	Gerar e verificar atestado
Execução velha perde utilidade após novo commit	Concorrência com cancelamento adequado ao processo

Laboratórios guiados

7.1 Laboratório de pipeline: matriz, outputs, artefatos e falha controlada

Objetivo: acompanhar o caminho de um dado e de um arquivo entre jobs, identificar a variante com falha e interpretar um resumo.

Pré-requisitos: um repositório de laboratório no GitHub.com com Actions habilitado, branch padrão `main` e permissão para adicionar workflows. Não são necessários serviços de nuvem nem segredos. O exemplo gera seus próprios arquivos e não depende de uma aplicação Node existente.

Crie `.github/workflows/laboratorio.yml`:

```
name: Laboratório GH-200

on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      forcar_falha:
        description: Fazer a variante Node 22 falhar
        type: boolean
        default: false

permissions:
  contents: read

concurrency:
  group: laboratorio-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  preparar:
    runs-on: ubuntu-24.04
    timeout-minutes: 5
    outputs:
      versao: ${ steps.meta.outputs.versao }
    steps:
      - id: meta
        run: |
          printf 'versao=lab-%s\n' "$GITHUB_RUN_NUMBER" >> "$GITHUB_OUTPUT"

  testar:
    name: Testar Node ${ matrix.node }
    needs: preparar
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    strategy:
      fail-fast: false
      max-parallel: 2
      matrix:
        node: ['22', '24']
    steps:
      - uses: actions/setup-node@v7
        with:
          node-version: ${ matrix.node }
          package-manager-cache: false
```

```

- name: Criar evidência
  env:
    VERSAO: ${needs.preparar.outputs.versao}
  run: |
    mkdir -p relatorio
    printf 'Versão: %s\n' "$VERSAO" > relatorio/resultado.txt
    node --version >> relatorio/resultado.txt

- name: Executar verificação
  id: teste
  env:
    FORCAR_FALHA: ${github.event_name == 'workflow_dispatch' && inputs.forcar_falha &&
matrix.node == '22' }
  run: |
    node <<'JS'
    const assert = require('node:assert/strict');
    const resultado = process.env.FORCAR_FALHA === 'true' ? 5 : 4;
    assert.equal(resultado, 2 + 2);
    console.log('Verificação concluída.');
```

JS

```

- name: Registrar resultado do teste
  if: ${!cancelled()}
  env:
    RESULTADO: ${steps.teste.outcome}
  run: |
    printf 'Resultado: %s\n' "$RESULTADO" >> relatorio/resultado.txt
    cat relatorio/resultado.txt >> "$GITHUB_STEP_SUMMARY"

- name: Preservar evidência
  if: ${!cancelled()}
  uses: actions/upload-artifact@v7
  with:
    name: relatorio-node-${matrix.node}
    path: relatorio/resultado.txt
    if-no-files-found: error
    retention-days: 7

resumir:
  needs: [preparar, testar]
  if: ${!cancelled()}
  runs-on: ubuntu-24.04
  timeout-minutes: 5
  steps:
    - uses: actions/download-artifact@v8
      with:
        pattern: relatorio-node-*
        path: resultados

    - name: Consolidar
      env:
        VERSAO: ${needs.preparar.outputs.versao}
        RESULTADO_MATRIZ: ${needs.testar.result}
      run: |
        printf '## Laboratório %s\n\n' "$VERSAO" >> "$GITHUB_STEP_SUMMARY"
        printf 'Resultado agregado: %s\n\n' "$RESULTADO_MATRIZ" >> "$GITHUB_STEP_SUMMARY"
        for arquivo in resultados/*/resultado.txt; do
          [ -f "$arquivo" ] || continue
          printf '### %s\n\n' "$arquivo" >> "$GITHUB_STEP_SUMMARY"
          cat "$arquivo" >> "$GITHUB_STEP_SUMMARY"
          printf '\n\n' >> "$GITHUB_STEP_SUMMARY"
        done

```

São **três definições de job e quatro jobs concretos**: um de preparação, dois de teste e um de resumo. O job de resumo consulta o resultado agregado da matriz; cada variante tem seu próprio arquivo de evidência. Não se tenta usar um único output de matriz como uma coleção determinística de todos os resultados.

Experimento	Resultado esperado	O que explicar
Execução manual sem falha	Duas variantes aprovadas e dois artefatos	Expansão da matriz e passagem de output
Execução manual com falha	Node 22 falha; Node 24 continua	Efeito de <code>fail-fast: false</code>
Examinar a execução com falha	Evidência da variante com erro preservada	Função de status nos steps posteriores
Examinar o resumo	Resultado agregado indica falha	Resumir não transforma teste com erro em sucesso
Alterar o nome de um artefato	Padrão de download precisa continuar compatível	Contrato entre produtor e consumidor
Remover <code>needs: preparar de testar</code>	O relacionamento de dados deixa de estar correto	<code>needs</code> é dependência e interface de outputs

Depois do teste com falha intencional, inicie uma **nova execução** com a opção desmarcada. Reexecutar a execução anterior mantém o evento e seus inputs; a falha intencional continuará sendo solicitada.

7.2 Laboratório de reutilização

Objetivo: transformar conhecimento sobre `workflow_call` em um contrato que outra pessoa consegue consumir.

- .. Crie os dois arquivos `reutilizavel.yml` e `chamador.yml` apresentados no domínio 2.
- !.. Execute o chamador e localize o output final.
- }. Altere o nome do output somente no produtor e observe a quebra do consumidor.
- }. Corrija o contrato em ambos os lados.
- }. Desenhe a chamada a um repositório central com SHA, sem publicar nada para realizar o exercício.

Critério de conclusão: explicar por que `uses` está no nível de job e identificar cada mapeamento que leva o valor do step interno ao `needs` do chamador.

Extensão: acrescente um input booleano e um segredo opcional de laboratório. Documente como o chamador os passa e o que acontece se não os fornecer. Não use credenciais reais para verificar apenas a passagem de valores.

7.3 Laboratório de autoria de ação

Objetivo: testar a ação composta e a JavaScript do domínio 3.

Caso	Entrada	Resultado esperado
Composta válida	<code>gh200</code>	Output começando com <code>gh200-</code>
Composta inválida	<code>nome com espaco</code>	Falha de validação
JavaScript válida	<code>loja-api</code>	Output <code>LOJA-API</code>
JavaScript vazia	String vazia	Falha com explicação
JavaScript excessiva	Mais de 40 caracteres	Falha de tamanho/formato

Copie o workflow de teste da ação composta e adapte o caminho da ação e o nome do input para testar a JavaScript. Quebre o nome `index.cjs` em `runs.main`, observe o erro de carregamento e restaure o caminho correto.

Critério de conclusão: distinguir falha de contrato, falha de empacotamento e falha de lógica. Uma ação pode ter YAML válido e ainda não conter o arquivo que o runner precisa executar.

7.4 Laboratório de governança

Objetivo: desenhar a configuração para dois repositórios e dois ambientes sem depender de acesso administrativo real.

Preencha esta tabela antes de alterar uma organização:

Recurso	Quem administra?	Quem consome?	Escopo necessário	Evidência a conferir
Workflow central				Referência e política de acesso
Variável compartilhada				Repositórios selecionados
Segredo de produção				Ambiente e proteções
Runner de produção				Grupo, rede e workflow autorizado
Política de ações				Origens e referências aceitas
Papel federado na nuvem				Claims aceitos e permissões do papel

Critério de conclusão: demonstrar que um PR externo pode ser testado sem acessar segredos e rede de produção. Em seguida, explicar como um commit aprovado chega à implantação com revisão e identidade adequadas.

7.5 Laboratório de revisão de segurança e eficiência

Uma equipe apresenta este cenário: o workflow reage a PRs, usa runner interno persistente, imprime o payload completo do evento, publica com PAT amplo, usa ações em `@main` e executa 18 variantes de teste em todo commit.

Faça uma revisão propondo uma alteração e uma justificativa para cada risco:

Observação	Ajuste esperado	Evidência de que melhorou
Código de contribuição em runner interno	Separar capacidade e confiança	Testes externos sem acesso à rede sensível
Payload completo no log	Registrar somente campos necessários	Diagnóstico útil sem exposição desnecessária
PAT amplo	Token com escopo mínimo ou OIDC, conforme o destino	Operação funciona com privilégio limitado
Ações em <code>@main</code>	Revisar e fixar referências	Alterações passam por revisão
18 variantes em todo evento	Relacionar cada combinação à cobertura exigida	Menos trabalho sem eliminar testes necessários
Evidência de build sem validação de origem	Verificar identidade, referência e proveniência	Implantação rejeita produtor inesperado

Critério de conclusão: justificar cada escolha pelo fluxo de confiança e pelo requisito do projeto. “É uma boa prática” não explica se a mudança realmente resolve o cenário.

Consulta rápida e pegadinhas

8.1 Números e limites

Os valores abaixo representam a documentação consultada, não promessas permanentes da plataforma.

Item	Valor ou regra	Fonte
Matriz	Até 256 jobs por execução de workflow	Limites
Job em runner hospedado pelo GitHub	Até 6 horas	Limites
Job auto-hospedado	Até 5 dias de execução	Limites
Execução completa do workflow	Até 35 dias, incluindo espera e aprovação	Limites
Espera por aprovação de ambiente	Até 30 dias	Limites
Job auto-hospedado em fila	Até 24 horas	Limites
Janela para reexecutar	Até 30 dias após a execução inicial	Reexecução
Intervalo de cron	Mínimo de 5 minutos	Agendamento
Logs e artefatos	Padrão de 90 dias; público: 1–90; privado: 1–400, sujeitos às políticas	Configuração
Tamanho de segredo	Até 48 KB	Segredos
Encadeamento de workflows reutilizáveis	Até 10 níveis, incluindo o chamador inicial	Reutilização
Workflows reutilizáveis únicos chamados	Até 50 a partir de um arquivo, considerando chamadas aninhadas	Reutilização
Validade do <code>GITHUB_TOKEN</code>	Termina com o job ou em 24 horas, o que vier primeiro	Autenticação automática
Token de instalação para ações de repositório privado	Escopo de leitura, expira em 1 hora	Compartilhar ações
Excluir uma execução	Concluída ou com mais de 2 semanas	Excluir execução
Job rejeitado na revisão de ambiente	O workflow falha	Revisar implantações

O limite do job auto-hospedado não prolonga a validade do `GITHUB_TOKEN`. Um job de longa duração precisa de uma estratégia de autenticação compatível com o tempo efetivo da operação.

O roteiro original citava um intervalo específico de long polling. Para configurar e diagnosticar runners, priorize requisitos documentados de conexão de saída, endpoints e disponibilidade; um intervalo interno de polling não deve ser tratado como garantia de protocolo para a arquitetura.

8.2 Sintaxe que precisa reconhecer

Necessidade	Sintaxe
Eventos	<code>on:</code>
Atividades de um evento	<code>types:</code>
Dependências de jobs	<code>needs:</code>

Necessidade	Sintaxe
Condição	<code>if:</code>
Chamada de ação ou workflow	<code>uses:</code> no nível correspondente
Parâmetros de uma chamada	<code>with:</code>
Implementação da ação	<code>runs.using</code>
Entrada do JavaScript	<code>runs.main</code>
Steps da composta	<code>runs.steps</code>
Segredo	<code>\${{ secrets.NOME }}</code>
Variável de configuração	<code>\${{ vars.NOME }}</code>
Output de step	<code>\${{ steps.identificador.outputs.nome }}</code>
Output entre jobs	<code>\${{ needs.identificador.outputs.nome }}</code>
Referência completa de branch	<code>refs/heads/main</code>
Referência completa de tag	<code>refs/tags/v1.0.0</code>
Negação explícita	<code>\${{ !cancelled() }}</code>

Uma ação remota usa uma referência, como `actions/checkout@v7`. Uma ação local, como `./.github/actions/identificacao`, usa outro formato e não recebe `@ref` dessa maneira. Não aplique a regra das ações remotas indistintamente aos caminhos locais.

8.3 Diferenças que mudam a resposta

Par	Diferença decisiva
Cache × artefato	Reaproveitamento de trabalho × resultado armazenado da execução
<code>env</code> × <code>vars</code>	Ambiente declarado para execução × configuração mantida no GitHub
<code>vars</code> × <code>secrets</code>	Configuração não sensível × dados sensíveis com controle específico
<code>GITHUB_ENV</code> × <code>GITHUB_OUTPUT</code>	Ambiente para próximos steps × resultado nomeado de um step
Label × runner group	Seleção de capacidade × controle de acesso
Matriz × concorrência	Quantidade de variantes × coexistência de execuções
<code>fail-fast</code> × <code>continue-on-error</code>	Cancelamento de variantes × tolerância à falha configurada
<code>outcome</code> × <code>conclusion</code>	Resultado anterior × posterior a <code>continue-on-error</code>
<code>github.actor</code> × <code>github.triggering_actor</code>	Ator associado à execução × pessoa que iniciou a tentativa
Runtime da ação × runtime do projeto	Node usado para executar a ação × Node instalado para a aplicação
Workflow reutilizável × composta	Jobs completos × steps dentro de um job
Workflow desabilitado × arquivo removido	Configuração reversível de execução × remoção da definição
Aprovar execução de fork × aprovar ambiente	Permitir execução do código × liberar implantação protegida
SHA × confiança	Identificação de código específico × avaliação de quem controla e do que faz esse código

8.4 Afirmações que exigem contexto

Afirmação simplificada	Interpretação correta
" <code>schedule</code> sempre usa UTC"	UTC é o padrão; a documentação atual admite <code>timezone</code>
"Uma tag de versão é imutável"	Tags comuns podem mudar; SHA e releases imutáveis têm propriedades próprias
"Toda ação do Marketplace foi revisada pelo GitHub"	A listagem não é uma auditoria do código
" <code>id-token: write</code> permite implantar na nuvem"	A permissão só permite solicitar identidade; o provedor aplica confiança e autorização
"Um segredo sempre está disponível se existe nas configurações"	Evento, política, escopo e ambiente afetam sua disponibilidade
"Environment sempre vence imediatamente em qualquer contexto"	Variáveis desse escopo têm momento de disponibilidade específico
"Um runner próprio é gratuito e seguro por estar dentro da empresa"	Há custos de operação e riscos de executar código com acesso à rede
"Fixar o SO congela todas as ferramentas"	Imagens ainda recebem atualizações
"Reexecutar testa a correção recém-enviada"	A reexecução mantém o commit/evento originais
" <code>max-parallel</code> diminui a quantidade de testes"	Diminui a simultaneidade, não a expansão da matriz
"Permitir somente a minha organização inclui o GitHub"	Ações de <code>actions/*</code> pertencem a outra organização
"YAML válido garante workflow válido"	O schema e as regras do Actions também precisam ser atendidos

8.5 Comandos de workflow

Comando ou arquivo	Finalidade
<code>::debug::mensagem</code>	Mensagem de debug, visível com debug de steps habilitado
<code>::notice::mensagem</code>	Anotação informativa
<code>::warning::mensagem</code>	Anotação de aviso
<code>::error::mensagem</code>	Anotação de erro; controle também o código de saída
<code>::add-mask::valor</code>	Registrar valor para mascaramento
<code>::group::título e ::endgroup::</code>	Agrupar linhas do log em uma seção recolhível, para organizar saída extensa
<code>::stop-commands::token</code>	Suspender o processamento de comandos de workflow, permitindo registrar texto sem executá-lo como comando
<code>GITHUB_ENV</code>	Persistir ambiente entre steps do mesmo job
<code>GITHUB_PATH</code>	Acrescentar diretório ao PATH dos próximos steps
<code>GITHUB_OUTPUT</code>	Definir output
<code>GITHUB_STEP_SUMMARY</code>	Acrescentar Markdown ao resumo

Uma anotação `::error::` não substitui necessariamente `exit 1` ou o tratamento de falha da implementação. O resultado do processo e as regras do workflow determinam a conclusão do step.

Índice de temas dos simulados

Use esta tabela quando uma questão dos [[Simulado 01|simulados]] deixar dúvida: localize o tema e vá direto à seção correspondente.

Tema	Onde consultar
Agendamento, cron, dias úteis	[[#Agendamento]]
Ambientes: revisores, auto-aprovação, rejeição, prazo de 30 dias	[[#Ambientes e aprovações]]
Âncoras, aliases, tabulação em YAML	[[#Âncoras, aliases e mesclagem YAML]]
Artefatos: upload, download, retenção, expiração, exclusão	[[#Cache, artefatos e resumos]]
Badge de status e seus parâmetros	[[#Cache, artefatos e resumos]]
Cache: chave, <code>restore-keys</code> , cache miss	[[#Cache, artefatos e resumos]]
<code>chmod</code> , permissão de execução, <code>entrypoint.sh</code>	[[#Scripts do repositório]]
Checks API, check suite, check run	[[#Anatomia da página de execução]]
Composta × reutilizável × template	[[#Escolher a forma de reutilização]]
Contextos e momento de avaliação	[[#Contextos e momento da avaliação]]
Debug logging, <code>ACTIONS_STEP_DEBUG</code>	[[#Logs, debug e reexecução]]
Desabilitar × excluir workflow × excluir execução	[[#Desabilitar, excluir arquivo e excluir execução]]
Diretório de workflows e de ações	[[#Primeiro workflow completo]], [[#Escolher o tipo de ação]]
Docker: só Linux, <code>Dockerfile</code> , <code>args</code> , <code>env</code>	[[#Ações Docker]]
Eventos, <code>types</code> , filtros de branch e path	[[#Eventos e filtros]]
Exit code, falha e sucesso de uma ação	[[#Dependências, condições e falhas]]
<code>GITHUB_ENV</code> , <code>GITHUB_PATH</code> , <code>GITHUB_OUTPUT</code> , <code>GITHUB_STEP_SUMMARY</code>	[[#Troca de dados]]
<code>GITHUB_TOKEN</code> : uso, permissões, validade	[[#Permissões do GITHUB_TOKEN]]
<code>INPUT_</code> , convenção de nomes de entrada	[[#Estrutura de metadados]]
Jobs dependentes, <code>needs</code> , <code>if</code> , job pulado	[[#Dependências, condições e falhas]]
Labels, grupos, grupo padrão, chargeback	[[#Labels e grupos]]
Logs: buscar, linkar linha, excluir, agrupar	[[#Anatomia da página de execução]], [[#Comandos de workflow]]
Marketplace: requisitos, categoria, Developer Agreement, verified creator	[[#Testar e distribuir]]
Matrizes: combinações, limite de 256, contexto <code>matrix</code>	[[#Matrizes]]
Metadados da ação: <code>action.yml</code> , <code>runs.using</code> , <code>outputs</code>	[[#Estrutura de metadados]]
OIDC, <code>id-token: write</code> , credenciais de longa duração	[[#OIDC para credenciais temporárias]]
Página da execução: filtros, "Set up job", abas	[[#Anatomia da página de execução]]
Políticas de enterprise e organização, ações permitidas	[[#Governança em camadas]]
Proxy, <code>--check</code> , status do runner, atualização automática	[[#Operação, rede e imagens]]
Publicar imagem em <code>ghcr.io</code> ou pacote npm	[[#Publicar pacotes e imagens]]
README de uma ação	[[#Testar e distribuir]]

Tema	Onde consultar
Reexecutar jobs com falha	[[#Logs, debug e reexecução]]
Runners: hospedado × auto-hospedado, quando usar cada um	[[#Runners hospedados e auto-hospedados]]
Segredos: escopos, precedência, limite de 48 KB, GPG, mascaramento	[[#Segredos, variáveis e escopo]], [[#Uso de segredos]]
Serviços em contêiner, ciclo de vida	[[#Jobs e serviços em contêiner]]
Templates da organização, repositório <code>.github</code> , nomenclatura	[[#Templates organizacionais]]
Troubleshooting por sintoma	[[#Tabela de diagnóstico]]
Variáveis padrão: <code>RUNNER_OS</code> , <code>GITHUB_REPOSITORY</code> , <code>GITHUB_ACTIONS</code>	[[#Variáveis de ambiente padrão]]
Variáveis de configuração <code>vars</code> e precedência	[[#Segredos, variáveis e escopo]]
Versionamento, SemVer, tags móveis, SHA completo	[[#Versionamento, releases e imutabilidade]]
Workflow reutilizável: contrato, <code>workflow_call</code>	[[#Contrato de um workflow reutilizável]]
<code>workflow_dispatch</code> , entradas obrigatórias, REST API e PAT	[[#Execução manual e entradas tipadas]]

Se o tema não estiver aqui, procure primeiro em [[#Consulta rápida e pegadinhas]] e depois na documentação oficial listada no fim deste material.

Simulado autoral — 25 questões

Responda sem consultar os capítulos. Cada questão informa quando há mais de uma alternativa correta. Para o diagnóstico deste material, conte uma questão de múltipla resposta como correta apenas se selecionar exatamente o conjunto esperado; essa é uma regra de estudo, não uma descrição do algoritmo de pontuação do exame oficial.

10.1

Questões do domínio 1

Q01. Um step escreve `CANAL=teste` em `GITHUB_ENV`. Onde o valor fica disponível automaticamente?

- A. Em todos os jobs do workflow.
- B. Nos próximos steps do mesmo job.
- C. Em todos os workflows do repositório.
- D. Apenas em workflows reutilizáveis.

Q02. Uma matriz possui três sistemas e duas versões. Uma combinação é removida por `exclude` e uma entrada de `include` só acrescenta uma propriedade a uma combinação existente. Quantos jobs a matriz gera?

- A. Quatro.
- B. Cinco.
- C. Seis.
- D. Sete.

Q03. Um job executa em um contêiner e usa um serviço chamado `banco`. Qual endereço representa o padrão apropriado para acessar esse serviço na rede compartilhada?

- A. `localhost`, necessariamente.
- B. O nome do repositório.
- C. O hostname `banco` e a porta do serviço.
- D. O IP público do runner, necessariamente.

Q04. Um input manual `publicar` foi declarado como booleano. Qual condição evita tratá-lo desnecessariamente como string?

- A. `if: ${{ inputs.publicar }}`.
- B. `when: publicar`.
- C. `if: publicar == true` sem contexto.
- D. `types: [publicar]`.

Q05. Você precisa passar uma versão curta e um pacote de build para um job dependente. Selecione **DUAS** escolhas adequadas.

- A. Output do job para a versão.
- B. Artefato para o pacote.
- C. `GITHUB_ENV` como transporte automático entre jobs.
- D. Gravar ambos somente no disco do primeiro runner.

10.2

Questões do domínio 2

Q06. Uma equipe quer centralizar três jobs que selecionam runners diferentes. Qual mecanismo atende melhor?

- A. Ação composta.
- B. Template que cada consumidor copia para sempre.
- C. Workflow reutilizável.
- D. Uma variável de organização.

Q07. O YAML foi corrigido em um novo commit. A pessoa reexecuta a execução anterior e observa o mesmo erro. Qual explicação deve ser investigada primeiro?

- A. Reexecuções ignoram todas as permissões.
- B. A reexecução mantém o commit e a referência originais.
- C. Ações nunca podem ser atualizadas.
- D. O runner sempre preserva todo o disco anterior.

Q08. Uma variante de matriz falha e o restante é cancelado. Qual opção ajuda a obter os resultados das demais variantes em uma próxima execução?

- A. `strategy.fail-fast: false`.
- B. Remover os nomes dos jobs.
- C. Trocar outputs por secrets.
- D. Aumentar `retention-days`.

Q09. O workflow reutilizável gera um valor em um step, mas o chamador não o recebe. Selecione TRÊS mapeamentos que devem ser conferidos após o step escrever em `GITHUB_OUTPUT`.

- A. Output do job chamado.
- B. Output em `on.workflow_call.outputs`.
- C. Acesso por `needs.<job>.outputs` no chamador.
- D. Publicação obrigatória no Marketplace.
- E. Criação obrigatória de um PAT.

Q10. Um workflow precisa ser suspenso temporariamente, preservando uma forma direta de reativá-lo. Qual operação corresponde ao objetivo?

- A. Excluir todos os artefatos.
- B. Remover todas as branches.
- C. Desabilitar o workflow.
- D. Reduzir o número de variantes.

10.3

Questões do domínio 3

Q11. Qual combinação de metadados identifica uma ação JavaScript do exemplo deste material?

- A. `runs.using: node24` e `runs.main`.
- B. `runs.using: composite` e `runs.image`.
- C. `jobs` e `workflow_call`.
- D. `container` e `services`.

Q12. Uma ação composta local não é encontrada. O diretório está correto no repositório. Qual passo pode estar faltando no job?

- A. Fazer checkout antes de chamar a ação.
- B. Publicar um release no Marketplace.
- C. Conceder `id-token: write`.
- D. Criar um runner group.

Q13. Uma ação JavaScript funciona na máquina do autor, mas o consumidor recebe erro de módulo ausente. Qual causa é mais plausível?

- A. O workflow tem um nome.
- B. As dependências necessárias não foram incluídas na distribuição.
- C. O repositório usa uma tag de versão.
- D. O consumidor não criou uma variável de ambiente de produção.

Q14. Qual referência identifica diretamente o commit revisado de uma ação remota?

- A. `@main`.
- B. `@latest`.
- C. `@v1`, independentemente da política do mantenedor.
- D. `@SHA_COMPLETO` verificado no repositório da ação.

Q15. Selecione DUAS afirmações corretas.

- A. Uma ação Docker exige ambiente Linux com Docker para a execução suportada descrita.
- B. Toda ação do Marketplace passa por auditoria de código do GitHub.
- C. Ação composta pode agrupar steps com `run` e `uses`.
- D. Declarar `required: true` elimina a necessidade de validar entradas na implementação.

Q16. A equipe precisa controlar quais repositórios acessam runners com conexão à produção. Qual recurso responde diretamente a essa necessidade?

- A. Nome do job.
- B. Runner group e sua política de acesso.
- C. Cache de dependências.
- D. `GITHUB_STEP_SUMMARY`.

Q17. Uma região de nuvem, sem conteúdo sensível, precisa ser configurável em vários repositórios selecionados. Qual opção é apropriada?

- A. Variável de organização com acesso aos repositórios necessários.
- B. Publicar um PAT dentro do YAML.
- C. Usar o token de registro do runner como nome da região.
- D. Criar uma ação Docker para cada região.

Q18. A política permite somente ações da organização `minha-empresa`. `actions/checkout` é bloqueada. Qual explicação é correta?

- A. `checkout` não é uma ação.
- B. A ação pertence à organização `actions`, fora do conjunto permitido.
- C. Toda ação pública é proibida por definição.
- D. O problema sempre é falta de `contents: write`.

Q19. Qual alternativa distingue corretamente o template organizacional?

- A. Fica obrigatoriamente em cada `node_modules`.
- B. Usa `workflow-templates/` na raiz do repositório organizacional chamado `.github`.
- C. Precisa ser uma imagem Docker.
- D. É atualizado automaticamente em todas as cópias existentes.

Q20. Um job está em fila e nenhum runner acessível possui todos os requisitos de seleção. Selecione DUAS verificações úteis.

- A. Labels e grupo solicitados por `runs-on`.
- B. Disponibilidade, conectividade e acesso aos runners compatíveis.
- C. Cor do badge do README.
- D. Texto do output de um step que ainda não executou.

10.5

Questões do domínio 5

Q21. Qual é o efeito de `id-token: write`?

- A. Conceder administração da conta de nuvem.
- B. Permitir solicitar uma identidade OIDC que poderá ser validada pelo provedor.
- C. Conceder escrita em todos os repositórios.
- D. Tornar qualquer PR confiável.

Q22. Um título de PR precisa ser impresso por um script. Qual padrão reduz o risco de injeção nessa operação?

- A. Inserir o título diretamente no código do `run`.
- B. Usar `eval` para interpretar o título.
- C. Passá-lo por `env` e imprimir com `printf '%s\n' "$TITULO"`.
- D. Conceder permissão de escrita ao token.

Q23. Um consumidor precisa comprovar que o arquivo recebido foi produzido pelo workflow esperado. Qual recurso atende ao requisito de origem?

- A. Apenas o nome do arquivo.
- B. Um badge verde sem identificar a execução.
- C. Um atestado de proveniência verificado com a identidade esperada.
- D. Um cache com chave baseada somente no sistema operacional.

Q24. Uma matriz tem dez jobs, cada um com três minutos. Todos conseguem executar em paralelo. Qual conclusão é correta?

- A. O trabalho agregado passa a ser de três minutos.
- B. O trabalho agregado é de aproximadamente 30 minutos, embora o tempo de espera possa se aproximar de três, mais overhead.
- C. Paralelismo elimina cobrança em qualquer plano.
- D. `max-parallel` apaga automaticamente combinações da matriz.

Q25. Um job privilegiado recebe um artefato produzido por um PR externo. Selecione DUAS atitudes apropriadas.

- A. Executar qualquer script do artefato porque foi armazenado no GitHub.
- B. Verificar a origem e a identidade do produtor antes do consumo sensível.
- C. Preservar a separação entre conteúdo não confiável e credenciais privilegiadas.
- D. Substituir o token por um PAT com todos os escopos.

10.6

Gabarito comentado

Ques-tão	Respos-ta	Explicação
Q01	B	<code>GITHUB_ENV</code> atende os próximos steps do mesmo job; não distribui estado entre runners.
Q02	B	São seis combinações iniciais, menos uma. A inclusão descrita apenas complementa uma combinação.
Q03	C	Na rede compartilhada dos contêineres, o serviço é acessado pelo nome e pela porta do serviço.
Q04	A	O contexto <code>inputs</code> preserva o booleano. <code>when</code> não é a chave da condição do Actions.
Q05	A, B	Outputs transportam pequenos valores; artefatos transportam arquivos.
Q06	C	Workflows reutilizáveis coordenam jobs e runners; ações compostas ficam dentro de um job.
Q07	B	Reexecutar uma execução existente mantém sua referência e seu commit.
Q08	A	A opção evita o cancelamento das demais variantes por esse mecanismo de falha rápida.
Q09	A, B, C	Cada camada precisa expor e consumir o valor pelo contrato correspondente.
Q10	C	Desabilitar é a operação reversível voltada a suspender execuções.
Q11	A	JavaScript declara runtime e arquivo de entrada em <code>runs</code> .
Q12	A	O runner precisa dos arquivos locais antes de resolver a ação pelo caminho.
Q13	B	A distribuição deve conter o que é necessário à execução; o consumidor não instala dependências automaticamente.
Q14	D	O SHA completo seleciona o commit. Isso ainda exige revisão e manutenção das atualizações.
Q15	A, C	Docker tem exigências do host; composta encapsula steps. Marketplace e metadados não substituem revisão e validação.
Q16	B	Grupos controlam acesso. Labels ajudam a escolher capacidade, mas não concedem acesso por si só.

Ques- tão	Respos- ta	Explicação
Q17	A	É configuração não sensível compartilhada com escopo controlado.
Q18	B	A política se refere à organização proprietária da ação, não a uma noção genérica de ação confiável.
Q19	B	O repositório <code>.github</code> hospeda os templates; as cópias são independentes.
Q20	A, B	O roteamento depende de seleção, acesso e capacidade disponíveis antes da execução.
Q21	B	OIDC separa emissão de identidade, confiança do provedor e autorização do recurso.
Q22	C	O dado permanece em uma variável, sem ser interpolado como código do shell nessa operação.
Q23	C	A verificação associa o artefato à proveniência esperada; nome e badge não fazem essa ligação.
Q24	B	Paralelismo reduz tempo de espera, mas não a soma da execução. O preço depende de outras condições.
Q25	B, C	Armazenar conteúdo na plataforma não muda a confiança em quem o produziu.

10.7 Registro de erros

Data	Avaliação/questão	Domínio	Causa do erro	Regra corrigida	Evidência ou exercício	Revisado em

Classifique a causa: conceito, leitura do enunciado, escopo, sintaxe, regra desatualizada ou quantidade de alternativas. O objetivo é produzir uma ação de estudo específica.

Domínio	Simulado autoral	Avaliação oficial	Avaliação adicional 1	Avaliação adicional 2	Prioridade de revisão
Criar e gerenciar workflows					
Consumir e solucionar problemas					
Criar e manter ações					
Gerenciar para a organização					
Automação segura e otimizada					

Use as células para registrar **quantidade de erros**, mantendo o mesmo critério em todas as tentativas. A avaliação prática oficial complementa os exercícios deste material.

Revisão final e fontes

11.1 Checklist de prontidão

- [] Consigo criar um workflow com eventos, permissões, jobs e steps sem copiar uma solução inteira.
- [] Consigo prever o efeito de filtros de branch, caminhos e tipos de evento.
- [] Distingo o contexto de avaliação da expressão do ambiente do shell.
- [] Sei passar dados por arquivos de ambiente, outputs e artefatos.
- [] Consigo expandir uma matriz, aplicar `include / exclude` e estimar trabalho agregado.
- [] Sei escolher endereço e porta de um serviço em contêiner conforme a topologia do job.
- [] Distingo âncoras, aliases e intenção de mesclagem YAML.
- [] Localizo o primeiro erro relevante e explico quando reexecutar ou iniciar uma nova execução.
- [] Distingo template, workflow reutilizável e ação composta.
- [] Consigo criar `action.yml`, validar inputs, emitir outputs e distribuir o código executável.
- [] Sei escolher entre ações JavaScript, Docker e compostas.
- [] Sei explicar tags, SHA, releases e imutabilidade sem tratá-los como equivalentes.
- [] Consigo desenhar grupos de runners, políticas e compartilhamento privado.
- [] Distingo `env`, `vars` e `secrets`, incluindo seus escopos e momentos de disponibilidade.
- [] Sei limitar o `GITHUB_TOKEN` e justificar quando outra identidade é necessária.
- [] Consigo explicar a troca OIDC e as condições de confiança no provedor.
- [] Identifico injeção de script e consumo indevido de código ou artefatos não confiáveis.
- [] Sei configurar aprovação de ambiente e verificar proveniência de build.
- [] Consigo propor uma otimização mensurável sem eliminar controles necessários.
- [] Revisei os erros e conferi o programa oficial antes de agendar a prova.

11.2 Perguntas para revisão oral

- .. Por que um `export` em um step não resolve a troca de ambiente com o próximo?
- !.. Como um valor sai de um step dentro de um workflow reutilizável e chega ao chamador?
- !.. O que muda se o teste roda no host ou no contêiner do job ao acessar PostgreSQL?
- !.. Por que reduzir `max-parallel` não reduz automaticamente o custo total?
- !.. Em que uma ação composta se diferencia de um workflow reutilizável que só tem um job?
- !.. Por que uma ação pública pode ser bloqueada mesmo com o repositório acessível?
- !.. Quais permissões um job realmente precisa para ler código, publicar pacote e solicitar OIDC?
- !.. Como a identidade de um PR externo poderia alcançar um recurso de produção por uma configuração incorreta?
- !.. O que um atestado prova e o que ele não prova?
- !.. Como você detectaria que uma regra decorada de um simulado antigo deixou de ser válida?

11.3 Fontes oficiais para manutenção do material

Os links ao longo dos capítulos sustentam as regras específicas. Use este índice para revisar o conteúdo quando houver atualização do exame ou da plataforma.

Tema	Fonte
Programa, objetivos e pesos	Guia GH-200
Condições e agendamento	Certificação GitHub Actions
Exercícios oficiais de diagnóstico	Avaliação prática
Estrutura dos workflows	Sintaxe
Eventos e referências	Eventos
Dados e avaliação	Contextos, expressões, comandos
Reutilização	Conceitos, referência, implementação
Ações próprias	Metadados, JavaScript, composta
Distribuição	Marketplace, releases imutáveis
Runners	Referência de runners próprios, grupos, imagens
Configuração e segredos	Variáveis, segredos
Segurança	Uso seguro, GITHUB_TOKEN, OIDC
Proveniência	Atestados
Capacidade	Límites, concorrência
Administração por API	Permissões, segredos, variáveis, artefatos

11.4 Como atualizar este guia

Ao atualizar o conteúdo, confira primeiro o programa do exame e depois as referências dos recursos afetados. Registre a data da revisão, atualize exemplos e gabaritos relacionados e diferencie GitHub.com de GHES. Versões de ações, imagens de runners, regras de retenção e recursos de segurança exigem atenção especial.

Preserve uma explicação reproduzível para cada regra: uma referência oficial, um exemplo mínimo ou uma execução de laboratório identificada. Quando o programa da prova e a documentação de implementação não forem igualmente explícitos, descreva a diferença em vez de transformar uma suposição em regra absoluta.

Síntese de revisão

Reorganização dos pontos centrais do material para a última revisão.

- Leia eventos, filtros, dependências e condições antes de prever a execução.
- Escolha outputs para pequenos valores, artefatos para arquivos e cache para reaproveitar trabalho.
- Distinga templates, workflows reutilizáveis e ações compostas pelo que cada um encapsula.
- Administre runners, variáveis, segredos e políticas conforme o acesso necessário.
- Revise permissões, entradas não confiáveis, OIDC e proveniência antes de conceder acesso à implantação.
- Use os laboratórios e o simulado para identificar lacunas e explicar cada correção.